BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC PHENIKAA



Học phần: CSE703093 – An toàn phần mềm

BÁO CÁO BÀI TẬP LỚN

End-to-end testing cho hệ thống tính tiền giảng dạy

Người hướng dẫn: ThS. Vũ Quang Dũng

Sinh viên thực hiện: Nguyễn Minh Dương – MSSV: 23010441

Đỗ Hữu Ngọc – MSSV: 23010822

HÀ NỘI, THÁNG 9 NĂM 2026

Mục lục

1 Giới thiệu và Mục tiêu **6**

1.1 Bối cảnh 6

1.2 Mục tiêu cụ thể 6

1.3 Phạm vi 7

2 Đặc tả Yêu cầu (Specification) **7**

2.1 Bảng đặc tả yêu cầu chức năng và an toàn (Hierarchical Requirements) . 8

2.2 Yêu cầu phi chức năng (NFR) 10

2.3 Miền giá trị hợp lệ theo đặc tả nghiệp vụ 12

2.4 Ràng buộc thiết kế/công nghệ (CON-x) 13

2.5 Tham số đầu vào: bắt buộc, tùy chọn, và tham số tổng hợp 16

2.6 Quy trình tổng quát: từ tổ hợp tham số đến trạng thái và statement . . . 18

2.7 Ví dụ đầy đủ 8 bước: `calculate_e2e_salary()` 19

3 Mô tả Hệ thống và Chương trình Mục tiêu **25**

3.1 Kiến trúc tổng quan DuongUniversity 25

3.2 Mô tả 3 module mục tiêu 26

3.3 Sơ đồ quan hệ struct (ERD) 28

3.4 Sơ đồ luồng dữ liệu (DFD) 28

3.5 Attack surface 30

3.6 Kích thước/độ phức tạp 30

4 Phương pháp Thực hiện **30**

4.1 Chương 1 – Mô hình hóa Kripke & Kiểm chứng BMC 30

4.2 Chương 2 – Kiểm toán an toàn bộ nhớ 33

4.3 Chương 3 – Kiểm chứng hình thức SMT/Z3 33

4.4 Chương 4 – Dynamic Fuzzing 35

5 Kết quả và Phân tích **35**

5.1 Kết quả Chương 1 – BMC 36

5.2 Kết quả Chương 2 – 9 findings 37

5.3 Kết quả kiểm chứng NFR-1/NFR-2 (`nfr_check.py`) 39

5.4 Kết quả Chương 3 – SMT 39

5.5 Kết quả Chương 4 – Fuzzing 40

5.6 Bảng tổng hợp CWE và mức độ rủi ro (CVSS-lite) 41

5.7 Ma trận truy vết Test Case → Yêu cầu (IEEE 829) 43

6 Thảo luận và Hạn chế **46**

6.1 Giới hạn phương pháp 46

6.2 So sánh với lý thuyết đã học 47

6.3 Hướng cải thiện 48

7 Kết luận **48**

Danh mục hình ảnh

3.1 Kiến trúc tổng quan DuongUniversity Management System – tầng tính toán C (viền đỏ nét đứt) biên dịch độc lập với .NET, không được GC/bounds-checking của .NET bảo vệ 26

3.2 Sơ đồ quan hệ 3 struct `GiangVien`, `LopHocPhan`, `Config` và các trường liên quan tới CWE-120/416/476/190 28

3.3 Sơ đồ luồng dữ liệu qua 3 hàm chính, đánh dấu vị trí phát sinh CWE và kết quả đối chứng Z3/fuzzing 29

4.1 Sơ đồ chuyển trạng thái Kripke (7 trạng thái, đã tách S_{2A}/S_{2B}) và kết quả BMC $k = 5$ đầy đủ P_1–P_5 trên cả bản BUGGY và FIXED, đối chiếu trực tiếp với `kripke_bmc.py` và `logs/kripke_bmc.log` 32

5.1 Black-box vs. White-box: (a) thời gian thực thi và (b) số lần thử cần để phát hiện đủ lỗ hổng đã biết (số liệu khớp Bảng 5.3) 41

5.2 So sánh điểm CVSS-lite giữa 6 CWE của đề tài (số liệu khớp Bảng 5.4) 42

Danh mục bảng

2.1 **Bảng Đặc tả Toàn bộ Yêu cầu Hệ thống (Hierarchical Requirements Specification)** 8

2.2 Yêu cầu phi chức năng (Non-Functional Requirements) 11

2.3 Miền giá trị hợp lệ theo đặc tả nghiệp vụ 13

2.4 Ràng buộc thiết kế/công nghệ (CON-x) và vị trí tương ứng trong mã nguồn, khớp toàn bộ REQ-1–10 14

2.5 Phân loại tham số đầu vào: bắt buộc vs. tùy chọn 16

2.6 9 tham số đầu vào của `calculate_e2e_salary(LopHocPhan *lhp, Config *cfg)`, đối chiếu với code thật 20

2.7 Rời rạc hóa từng tham số (đã loại A_8 dead field) 21

2.8 Ràng buộc giữa các tham số, đối chiếu với code thật 22

2.9 15 statement đại diện (đã sửa ST-09/ST-10 theo đúng `heSoQuyMo` thật) . 23

5.1 Kết quả BMC $k = 5$ trên 5 thuộc tính P_1–P_5 36

5.2 Toàn bộ 9 cảnh báo phát hiện bởi `pattern_scanner.py` trên code VULN; `main_fixed.c` sạch tuyệt đối (0 finding) 37

5.3 So sánh Black-box vs. White-box (lần chạy đại diện, `logs/fuzzer_black/whitebox.log`) 40

5.4 Tổng hợp CWE, vị trí trong đề tài và điểm CVSS-lite tự tính 42

5.5 Ví dụ thực tế minh họa cho mỗi lớp CWE 43

5.6 Ma trận truy vết Test Case → Yêu cầu (mọi kết quả thực tế trích từ `logs/*.log` thật) 44

Danh mục mã nguồn

1 Struct `GiangVien` – trích nguyên văn `giang_vien.h` (dòng 7–14) . . . 27

2 Struct LopHocPhan – trích nguyên văn lop_hoc_phan.h (dòng 9–17) . 27

3 delete_lecturer_VULN() – CWE-416: free() không huỷ liên kết . . 37

4 delete_lecturer_SAFE() – khắc phục: nhận GiangVien ** để nullify con trỏ gốc 38

5 pack_class_size_byte_VULN() – CWE-197: ép kiểu không kiểm tra phạm vi 38

6 pack_class_size_byte_SAFE() – khắc phục: guard [0,127] trước khi ép kiểu 38

1 Giới thiệu và Mục tiêu

1.1 Bối cảnh

Các hệ thống quản lý đào tạo đại học quy mô lớn – như **DuongUniversity Management System** – xử lý đồng thời dữ liệu nhân sự (giảng viên), dữ liệu học vụ (lớp học phần) và dữ liệu tài chính (thù lao giảng dạy). Khi ba loại dữ liệu này liên thông với nhau qua các mô-đun C tốc độ cao (thường dùng ở tầng tính toán/tích hợp phía sau một backend .NET/PostgreSQL), một lỗi thao tác bộ nhớ ở mô-đun đầu (ví dụ giải phóng bộ nhớ giảng viên không đúng cách) có thể lan truyền thành lỗi tài chính nghiêm trọng ở mô-đun cuối (tính sai hoặc tính nhầm thù lao). Đây chính là rủi ro an toàn đặc trưng của hệ thống quản lý tài chính giáo dục: hậu quả của một lỗi an toàn bộ nhớ không dừng lại ở việc chương trình crash, mà có thể trở thành một quyết định tài chính sai (trả nhầm lương, từ chối duyệt lương cho giảng viên đã dạy thật).

Đề tài chọn đúng **mô-đun tính thù lao giảng viên** (`tinh_tien_day.c`, liên kết với `giang_vien.c` và `lop_hoc_phan.c`) làm đối tượng phân tích vì đây là nơi hội tụ đầy đủ các đặc điểm cần thiết để một bài tập lớn an toàn phần mềm có ý nghĩa thực chất: (1) **xử lý input người dùng trực tiếp** (họ tên, bằng cấp, mã lớp qua `stdin`); (2) **thao tác con trỏ xuyên mô-đun** (`LopHocPhan.gv_ptr` trỏ chéo sang `GiangVien`); và (3) **phép toán tài chính có thể tràn số** (nhân dồn số tiết, hệ số, đơn giá). Ba đặc điểm này tạo thành một *attack surface* rõ ràng, đủ nhỏ để phân tích triệt để bằng 4 kỹ thuật kiểm chứng khác nhau, nhưng đủ thật để không chỉ là bài tập minh họa.

1.2 Mục tiêu cụ thể

Bài tập lớn đặt ra 4 mục tiêu cụ thể, mỗi mục tiêu ứng với một chương kỹ thuật, áp dụng trọn vẹn khung 4 chương công cụ của học phần[15] lên đúng một hệ thống E2E thay vì các bài tập minh hoạ độc lập:

1. **Phát hiện lỗ hổng bộ nhớ và dữ liệu bằng phân tích tĩnh**: quét trực tiếp mã nguồn C, phát hiện tối thiểu 6 lớp lỗ hổng – **CWE-120** (Buffer Copy without Checking Size), **CWE-416** (Use-After-Free), **CWE-476** (NULL Pointer Dereference), **CWE-190** (Integer Overflow), **CWE-1288** (Improper Validation of Specified Quantity/Enum), và **CWE-197** (Numeric Truncation Error) – không cần biên dịch hay chạy chương trình.

2. **Kiểm chứng luồng nghiệp vụ bằng mô hình hóa hình thức**: dựng mô hình Kripke $M = (S, S_0, R, L)$ cho luồng E2E 3 mô-đun và kiểm chứng bằng Bounded Model Checking (BMC, $k = 5$) trên 5 thuộc tính LTL/CTL.

3. **Chứng minh (hoặc bác bỏ) an toàn số học trên toàn miền giá trị bằng SMT**: dùng Z3 solver chứng minh phép tính thù lao có/không thể tràn số 32-bit, và chứng minh hình thức (không chỉ test mẫu) rằng bản vá loại trừ hoàn toàn lỗi.

4. **Xác nhận lỗ hổng biểu hiện thật trên binary đã biên dịch bằng fuzzing động**: black-box (input ngẫu nhiên, đo crash rate thật trên `salary_calc.exe`) và white-box (Z3-guided, giải trực tiếp điều kiện gây lỗi).

1.3 Phạm vi

Sẽ thực hiện: phân tích, mô hình hóa và kiểm chứng 3 file module C – `giang_vien.c` (Module 1 – Quản lý Giảng viên), `lop_hoc_phan.c` (Module 2 – Quản lý Lớp học phần), `tinh_tien_day.c` (Module 3 – Tính tiền dạy & Thanh toán) – cùng 2 driver `main.c` (bản có lỗ hổng cố ý, VULN) và `main_fixed.c` (bản đã khắc phục, SAFE, dùng chung 3 module với bản VULN). Toàn bộ 4 chương công cụ (`ch1_models`, `ch2_memsafe`, `ch3_staticsat`, `ch4_dynafuzz`) đều áp dụng trên đúng 3 module này, đảm bảo tính nhất quán E2E thay vì 4 ví dụ minh họa rời rạc không liên quan tới nhau.

Sẽ không thực hiện: triển khai/kiểm chứng backend .NET 9 Web API, frontend React, hay kết nối cơ sở dữ liệu PostgreSQL thực của hệ thống DuongUniversity Management System – các thành phần này chỉ được mô tả ở mức kiến trúc tổng quan (Mục 3) để xác định đúng vị trí của 3 module C trong hệ thống thật, không nằm trong phạm vi kiểm chứng của đề tài.

2 Đặc tả Yêu cầu (Specification)

Trước khi phân tích lỗ hổng, đề tài đặc tả tường minh *vùng input hợp lệ* và các quy tắc an toàn mà 3 module bắt buộc phải tuân thủ. Đây là cơ sở để mọi kỹ thuật kiểm chứng ở Mục 4 trả lời cùng một câu hỏi cốt lõi: *"Mô-đun có chịu được khi input vượt ra ngoài vùng an toàn (đặc tả) hay không?"* – thay vì chỉ hỏi "output có đúng không" trên một vài trường hợp thử.

2.1 Bảng đặc tả yêu cầu chức năng và an toàn (Hierarchical Requirements)

Bảng 2.1 là bảng đặc tả toàn diện và duy nhất của đề tài, tích hợp phân cấp 2 tầng theo chuẩn kỹ thuật phần mềm: tầng tổng quan (**REQ-1** đến **REQ-10**, ứng với từng mục tiêu an toàn/CWE) và tầng chi tiết (**REQ-**$X.Y$, suy ra từ các statement điều kiện biên và rẽ nhánh của mã nguồn). Mỗi yêu cầu đều gắn trực tiếp với loại (Safety, Memory, Functional, Integrity), mức ưu tiên, statement gốc và chương/test case kiểm chứng tương ứng. Các yêu cầu phi chức năng (NFR) được đặc tả bổ trợ tại Bảng 2.2 ngay sau đó.

Bảng 2.1: **Bảng Đặc tả Toàn bộ Yêu cầu Hệ thống (Hierarchical Requirements Specification)**

Mã REQ	Nội dung đặc tả	Từ ST	Chương / Test case kiểm chứng
REQ-1 [Safety – HIGH]: **Không được tính tiền khi chưa phân công giảng viên**			
REQ-1.0	Chặn tính tiền khi lớp học phần chưa gán giảng viên	ST-01	Ch1 BMC P_1 (SAT trên BUGGY / UNSAT trên FIXED)
REQ-2 [Safety – HIGH]: **Không được xuất báo cáo khi chưa tính tiền**			
REQ-2.0	Báo cáo xuất ra bắt buộc phải có dữ liệu thù lao hợp lệ	ST-01	Ch1 BMC P_2 (UNSAT cả 2 bản)
REQ-3 [Memory – HIGH]: **Các buffer chuỗi không tràn bộ đệm khi input vượt giới hạn (CWE-120)**			
REQ-3.1	Buffer `hoTen[80]` không tràn khi input $>$ 80 ký tự	CON-1	Ch2 `strcpy`; Ch4 Target 1; Ch4 72/200 crash
REQ-3.2	Buffer `maLop[20]` không tràn khi input $>$ 20 ký tự	CON-4	Ch2 `strcpy`; Ch4 Target 2
REQ-3.3	Buffer `bangCap[20]`, `maGV[20]` an toàn (`snprintf`)	CON-2,3	Ch2 `strcpy` (`giang_vien.c:12,18`)
REQ-4 [Memory – HIGH]: **Con trỏ `gv_ptr` không truy cập sau khi `free()` (CWE-416 UAF)**			
REQ-4.1	`is_deleted=1` $\rightarrow$ trả về 0	ST-02	Ch3 Check 3 (UNSAT)
REQ-4.2	`is_deleted` phải kiểm tra TRƯỚC khi đọc `heSoBangCap`	ST-02	Ch3 Check 3; Ch2 dòng 34
REQ-4.3	Bản SAFE đặt `is_deleted=1` TRƯỚC `free()`	ST-03	Ch1 BMC P_1; Ch4 White-box Target 3

(tiếp theo Bảng 2.1 – Đặc tả toàn bộ yêu cầu hệ thống)

Mã REQ	Nội dung đặc tả	Từ ST	Chương / Test case kiểm chứng
REQ-4.4	`lhp->gv_ptr` phải NULL hóa TRƯỚC khi `free(gv)`	ST-03	Ch2 `dangling_free`; Ch2.1 ASan offset +24
REQ-5 [Memory – HIGH]: **Con trỏ `gv_ptr` phải được kiểm tra NULL trước khi đọc (CWE-476)**			
REQ-5.1	`gv_ptr=NULL` → trả về 0, không crash	ST-01	Ch2 `unchecked_nested_deref`; Ch4 Target 4
REQ-5.2	`gv_ptr=NULL` → không đọc field nào của `*gv_ptr`	ST-01	Ch2 `unchecked_nested_deref`
REQ-6 [Memory – HIGH]: **Phép nhân dồn không tràn số nguyên 32-bit (CWE-190)**			
REQ-6.1	Phép tính trung gian dùng `double`	ST-04	Ch3 Check 2
REQ-6.2	Kết quả cuối dùng `long long`	ST-04	Ch3 Check 2
REQ-6.3	GS $\times$ 500 $\times$ dmt_max không tràn `long long`	ST-04	Ch3 Check 2 (UNSAT)
REQ-6.4	Tổng $(\texttt{heSoHocPhan} + \texttt{heSoQuyMo}) \times \texttt{soTiet}$ tối đa không tràn `double`	ST-10	Ch3 Check 2 (UNSAT)
REQ-7 [Functional – MEDIUM]: **Công thức tính thù lao 2 bước đúng với mọi input hợp lệ**			
REQ-7.1	CN $\times$ 1 $\times$ dmt_min cho kết quả dương	ST-05	Ch3 Check 1/2
REQ-7.2	Kết quả tối thiểu > 0 với mọi đầu vào hợp lệ	ST-05	Ch3 Check 1/2
REQ-7.3	`soTiet` $\leq 0 \rightarrow$ kẹp về 0 an toàn	ST-07	Ch4 BVA (đề xuất bổ sung log cảnh báo)
REQ-7.4	`soTiet` $> 500 \rightarrow$ hiện chưa có guard C tường minh	ST-08	Ch3/Ch4 (giới hạn thật, chưa vá – Mục 6)
REQ-7.5	`soTiet` $= 500 \rightarrow$ tính đúng, không tràn	ST-08	Ch3 Check 2
REQ-7.6	Khi `soLuongSinhVien` ≤ 50: heSoQuyMo $= 0.0$	ST-09	Ch3 Check 1/2 (mô hình đã sửa)
REQ-7.7	Khi `soLuongSinhVien` > 50: heSoQuyMo $= 0.2$	ST-10	Ch2 dòng 9/39; Ch3 Check 1/2
REQ-8 [Integrity – HIGH]: **Đóng gói audit log không bị sai lệch kiểu (char) (CWE-197)**			

(tiếp theo Bảng 2.1 – Đặc tả toàn bộ yêu cầu hệ thống)

Mã REQ	Nội dung đặc tả	Từ ST	Chương / Test case kiểm chứng
REQ-8.1	`soLuongSinhVien` $\in [0, 127] \rightarrow$ byte chính xác	ST-11	Ch3 Check 5 (UNSAT); Ch4 BVA 17/17
REQ-8.2	`soLuongSinhVien` $= 128 \rightarrow$ SAFE bị chặn	ST-12	Ch3 Check 4/5; Ch4 BVA
REQ-8.3	`soLuongSinhVien` $> 127 \rightarrow$ ghi sentinel, không wrap-around	ST-12	Ch4 BVA (146/200 chặn an toàn)
REQ-8.4	Không tồn tại 2 `soLuongSinhVien` khác nhau cùng 1 byte	ST-13	Ch3 Check 6 (SAT trên VULN)
REQ-8.5	`soLuongSinhVien` $= 0$ và $= 256$ phải cho byte khác nhau	ST-13	Ch3 Check 6; Ch4 Target 7
REQ-9 [Functional – MEDIUM]: `bangCap` **phải thuộc danh mục hợp lệ (CWE-1288)**			
REQ-9.1	`bangCap` không hợp lệ phải bị chặn/cảnh báo tại `add_lecturer()`	ST-14	Ch3 Check 7 (SAT)
REQ-9.2	`calculate()` không được nhận `GiangVien` có `bangCap` sai	ST-14	Ch3 Check 7
REQ-10 [Memory – HIGH]: **Con trỏ** `cfg` **(**`Config*`**) phải kiểm tra NULL trước khi đọc**			
REQ-10.1	`cfg=NULL` $\rightarrow$ trả về -1	ST-06	Ch2 dòng 30; Ch4 BVA
REQ-10.2	Không đọc `dinhMucTienChuan` khi `cfg=NULL`	ST-06	Ch2 dòng 30

2.2 Yêu cầu phi chức năng (NFR)

Bảng 2.2: Yêu cầu phi chức năng (Non-Functional Requirements)

Mã NFR	Thuộc tính	Nội dung đặc tả	Đối chiếu kiểm chứng
NFR-1	Hiệu năng	Thời gian tính lương cho 1 lớp < 10 ms (không tính I/O) trên phần cứng phổ thông.	**Đã đo thực nghiệm** (subcommand `e2e_nfr_bench`, `main_fixed.c`/`main.c`, dùng `QueryPerformanceCounter` vì `clock()` trên Windows chỉ có độ phân giải $\approx$15 ms – quá thô để đo 1 phép tính $O(1)$): 10 000 lần gọi `calculate_e2e_salary_SAFE()` mất tổng $0{,}141$ ms $\Rightarrow$ trung bình $0{,}000014$ ms/lần (bản VULN: $0{,}000011$ ms/lần) – thấp hơn ngưỡng gần 6 bậc độ lớn – **PASS** (`logs/nfr_check.log`).
NFR-2	Độ ổn định bộ nhớ	Không phát sinh lỗi bộ nhớ (`heap-use-after-free`, buffer overflow) khi lặp luồng E2E ≥ 10.000 lần.	**Đã đo thực nghiệm ở đúng quy mô** 10^4 (cùng subcommand `e2e_nfr_bench`, đo `WorkingSetSize` qua `GetProcessMemoryInfo` trước/sau 10 000 lần tạo/xoá `GiangVien+LopHocPhan`): drift $= 4$ KB trên bản SAFE, 4 KB trên bản VULN (ngưỡng < 1024 KB) – **PASS cả 2 bản** (`logs/nfr_check.log`), bổ trợ cho 0 vi phạm ASan (Mục 4.2) và 0/200 crash fuzzing bản FIXED (Mục 4.4) đã có trước đó.
NFR-3	Tính di động	Biên dịch sạch (0 warning với `-Wall`) trên ≥ 2 toolchain khác nhau.	Đã xác nhận biên dịch và chạy thành công trên GCC/MSYS2 UCRT64 và Clang64 (Mục 4.2) – **PASS**.
NFR-4	Chịu lỗi (Robustness)	Khi input vượt đặc tả (Bảng 2.3), bản vá phải từ chối có kiểm soát (trả mã lỗi), không crash.	0/200 crash trên fuzzing bản FIXED; 146/200 lần chặn an toàn đúng thiết kế trên kịch bản CWE-197 (Mục 5) – **PASS**.
NFR-5	Toàn vẹn (Audit)	Hai giá trị `soLuongSinhVien` khác nhau không bị map trùng 1 byte trong audit log (non-collision).	Check 6 (Mục 4.3) và Target 7 (Mục 4.4) tái hiện vi phạm trên VULN; bản FIXED chặn trước ép kiểu – **PASS trên FIXED**.

(tiếp theo Bảng 2.2)

Mã NFR	Thuộc tính	Nội dung đặc tả	Đối chiếu kiểm chứng
NFR-6	Tính tất định	Cùng bộ input, kết quả BMC/SMT và số liệu fuzzing phải nhất quán ở mọi lần chạy độc lập.	Ch1/Ch3: `random_seed` cố định trong `kripke_bmc.py`/Z3 – SAT/UNSAT và phản ví dụ giống nhau mọi lần chạy – **PASS**. Ch4 black-box: đã thêm `random.seed(42)` cho `fuzzer_blackbox.py` để đầu vào sinh ra tất định, nhưng **số crash vẫn dao động** (60–72/200 đo được qua nhiều lần chạy lại cùng seed) vì ASLR/heap-layout của Windows CRT nằm ngoài kiểm soát của seed Python (xem cơ chế tại Mục 6) – kết luận định tính (VULN có crash, FIXED 0 crash) tất định, nhưng số đếm chính xác thì **không** – **PASS một phần**, số liệu trích trong báo cáo là 1 lần chạy đại diện đã lưu log.
NFR-7	Không có dead field	Mọi field khai báo trong struct cấu hình (`Config`, ...) phải được ít nhất 1 hàm thực sự đọc – không giữ lại field không dùng, dễ gây hiểu sai khi mô hình hóa.	Rà soát toàn bộ `Config` (không chỉ theo lead của Check 1/2): **2/3 field không hề được đọc** – `heSoQuyMoLop` (`tinh_tien_day.h:8`) *và* `nam` (`tinh_tien_day.h:13`, chỉ gán lúc khởi tạo tại `main.c:15`/`main_fixed.c:16`, không hàm nào đọc lại) – **FAIL**, xem Mục 2.7. Ghi chú phụ: `id` trong `GiangVien`/`LopHocPhan` cũng write-only, nhưng thuộc lớp khác (định danh dự phòng, không phải tham số nghiệp vụ) nên không tính là FAIL của NFR này.

2.3 Miền giá trị hợp lệ theo đặc tả nghiệp vụ

Bảng 2.3 liệt kê chính xác miền giá trị hợp lệ theo đặc tả *nghiệp vụ* (không phải giới hạn kỹ thuật của kiểu dữ liệu C – xem phân biệt tại Mục 2.4) của DuongUniversity cho từng trường dữ liệu chịu trách nhiệm bởi REQ-3, REQ-6, REQ-8, REQ-9. Đây là các con số được dùng trực tiếp làm ràng buộc khi mô hình hóa Z3 SMT (Mục 4.3) và khi thiết kế bộ input biên (Boundary Value Analysis) cho fuzzing (Mục 4.4).

Bảng 2.3: Miền giá trị hợp lệ theo đặc tả nghiệp vụ

Trường dữ liệu	**Miền hợp lệ (nghiệp vụ)**	**YC liên quan**
`hoTen`	Chuỗi họ tên giảng viên hợp lệ	REQ-3
`maLop`	Chuỗi mã lớp học phần hợp lệ	REQ-3
`bangCap`	$\in$ {GS, PGS, TIEN_SI, THAC_SI, CU_NHAN}	REQ-9
`soTiet`	$[1, 500]$	REQ-6, REQ-7
`dinhMucTienChuan`	$[50\,000, 200\,000]$ VNĐ	REQ-6, REQ-7
`soLuongSinhVien`	$[0, 300]$ (lớp đại cương có thể tới vài trăm SV, đại diện cho BVA)	REQ-8
`gv_ptr`	Khác `NULL` và trỏ tới vùng nhớ còn hợp lệ trước khi đọc	REQ-4, REQ-5

Mọi giá trị nằm ngoài các miền trên đều là input hợp lệ về mặt *cú pháp* (compiler chấp nhận) nhưng vi phạm đặc tả nghiệp vụ – và chính xác đây là những trường hợp mà 4 chương kiểm chứng ở Mục 4 tập trung khai thác để trả lời câu hỏi "module có chịu được không". Lưu ý quan trọng: *miền nghiệp vụ* của `soLuongSinhVien` ($[0, 300]$, không có giới hạn kỹ thuật cứng) **khác hẳn** *giới hạn kỹ thuật* khi đóng gói giá trị đó xuống kiểu `char` ($[0, 127]$) – chính khoảng cách giữa hai miền này là nguồn gốc của CWE-197, được đặc tả tường minh ở CON-5 dưới đây.

2.4 Ràng buộc thiết kế/công nghệ (CON-x)

Khác với Bảng 2.3 (miền giá trị *nghiệp vụ* – do người phân tích đặc tả đặt ra), Bảng 2.4 liệt kê các **ràng buộc thiết kế/công nghệ** – giới hạn áp lên hệ thống bởi chính *cách hiện thực* (kích thước buffer khai báo trong struct C, phạm vi kiểu dữ liệu) chứ không phải bởi quy tắc nghiệp vụ. Việc tách hai loại ràng buộc này (trước đây bị trộn chung vào Bảng 2.3) là quan trọng vì phần lớn CWE trong đề tài phát sinh đúng tại *khoảng cách* giữa hai miền: nghiệp vụ cho phép một giá trị, nhưng công nghệ (kiểu dữ liệu/kích thước buffer) không đủ chỗ chứa giá trị đó.

Bảng 2.4 nay liệt kê **13 ràng buộc CON-1–13**, đủ để **mọi REQ-1 đến REQ-10** đều truy vết được về ít nhất 1 ràng buộc kỹ thuật/thiết kế cụ thể (bảng cũ chỉ phủ REQ-3, 6, 8, 9 – bỏ trống hoàn toàn REQ-1, 2, 4, 5, 7, 10). 7 ràng buộc mới (CON-7–13) không phải kích thước buffer mà là **ràng buộc kiểu dữ liệu và luồng điều khiển**: con trỏ `NULL`-able (CON-7, 8), quản lý bộ nhớ thủ công (CON-9), thiếu cờ trạng thái tường minh (CON-10), literal cứng thay cho hằng số có tên (CON-11), guard một chiều (CON-12), và chuỗi tự

do thay cho enum (CON-13) – tất cả đều đối chiếu trực tiếp với dòng code thật.

Bảng 2.4: Ràng buộc thiết kế/công nghệ (CON-x) và vị trí tương ứng trong mã nguồn, khớp toàn bộ REQ-1–10

Mã CON	**Ràng buộc kỹ thuật**	**Vị trí trong code**	**REQ liên quan**
CON-1	Buffer `hoTen` tối đa **80** byte (79 ký tự thật + `'\0'`)	`#define MAX_NAME 80` (`giang_vien.h:4`)	REQ-3.1 / CWE-120
CON-2	Buffer `maGiangVien` tối đa **20** byte (hằng số 20 trực tiếp, không qua macro riêng)	`char maGiangVien[20]` (`giang_vien.h:9`)	REQ-3.3 / CWE-120
CON-3	Buffer `bangCap` tối đa **20** byte (cùng kiểu hằng số trực tiếp như CON-2)	`char bangCap[20]` (`giang_vien.h:11`)	REQ-3.3 / CWE-120
CON-4	Buffer `maLop` tối đa **20** byte	`#define MAX_MALOP 20` (`lop_hoc_phan.h:6`)	REQ-3.2 / CWE-120
CON-5	Kiểu `char` (signed 8-bit) giới hạn an toàn $[0, 127]$ – vượt quá sẽ wrap-around	`pack_class_size_byte_*()` (`tinh_tien_day.c:67,75`)	REQ-8.1–8.5 / CWE-197
CON-6	Kiểu `int` 32-bit giới hạn kết quả phép nhân dồn về [INT_MIN, INT_MAX] (bản VULN); bản SAFE đổi sang `double`/`long long`	ép kiểu `(int)` trong `calculate_e2e_salary_VULN` (`tinh_tien_day.c:12-13`)	REQ-6.1–6.4 / CWE-190
CON-7	`gv_ptr` là con trỏ `GiangVien*` cho phép `NULL` (C không có kiểu non-null/`Option`) – trạng thái "đã phân công" chỉ được *suy ra ngầm* từ `gv_ptr != NULL`, không có cờ boolean riêng	`LopHocPhan.gv_ptr` (`lop_hoc_phan.h:15`)	REQ-1.0, REQ-5.1, REQ-5.2 / CWE-476
CON-8	`cfg` là con trỏ `Config*` cho phép `NULL`, cùng nguyên nhân kỹ thuật như CON-7	tham số `Config *cfg` trong `calculate_e2e_salary_*()` (`tinh_tien_day.h:17,20`)	REQ-10.1, REQ-10.2 / CWE-476
CON-9	`GiangVien` cấp phát bằng `malloc()`/giải phóng bằng `free()` thủ công – C không có RAII/GC/reference counting theo dõi vòng đời con trỏ dùng chung giữa 2 module	`add_lecturer_*()` (`malloc`); `delete_lecturer_VULN()` (`free`, `giang_vien.c:35`)	REQ-4.1–4.4 / CWE-416

(tiếp theo Bảng 2.4)

Mã CON	Ràng buộc kỹ thuật	Vị trí trong code	REQ liên quan
CON-10	Không có cờ trạng thái "đã tính lương" – `tienDay` truyền vào `export_report_*()` như 1 tham số độc lập; thứ tự `create`→`calculate`→`export` chỉ được đảm bảo bởi trình tự gọi hàm viết tay trong `main()`, không có kiểm tra bất biến runtime	tham số `tienDay` của `export_report_SAFE()` (`tinh_tien_day.h:23`); trình tự gọi tại `main_fixed.c:39-43`	REQ-2.0
CON-11	Ngưỡng quy mô lớp (50 sinh viên) là **literal cứng** ngay trong công thức, không phải `#define` hay field `Config` có tên	`(lhp->soLuongSinhVien > 50)` (`tinh_tien_day.c:9` VULN, `:41` SAFE)	REQ-7.6, REQ-7.7
CON-12	`soTiet` chỉ có guard *một chiều* (kẹp về 0 nếu ≤ 0); không có guard trên tương ứng miền nghiệp vụ `soTiet` ≤ 500 (Bảng 2.3)	`lhp->soTiet = (soTiet>0)?soTiet:0` (`lop_hoc_phan.c:31`) – không có nhánh nào chặn `soTiet>500`	REQ-7.3, REQ-7.4, REQ-7.5
CON-13	`bangCap` là `char[20]` tự do (chuỗi), không phải kiểu `enum` – tập giá trị hợp lệ {GS, PGS, TIEN_SI, THAC_SI, CU_NHAN} chỉ được ép bằng chuỗi `if/strcmp`, không có ràng buộc ở tầng kiểu dữ liệu (compile-time)	chuỗi `if(strcmp(bangCap,...))` trong `add_lecturer_VULN/SAFE` (`giang_vien.c:19-23`)	REQ-9.1, REQ-9.2 / CWE-1288

Đúng như quan sát ở Mục 5: kích thước buffer $\{80, 20, 20\}$ (CON-1, CON-3, CON-4 – ba trường được nêu trực tiếp trong REQ-3) khớp *tường minh, từng con số* với đúng 3 định nghĩa struct thật trong Listings 1 and 2 (Mục 3), không phải số minh họa; CON-2 (`maGiangVien[20]`) là buffer thứ 4 cũng bị `strcpy` tràn (`giang_vien.c:12` trong Bảng 5.2) nhưng chưa được liệt trong văn bản REQ-3 gốc – một khoảng hở đặc tả nhỏ được phát hiện chính nhờ việc tách bảng CON riêng và đối chiếu lại với kết quả quét Chương 2. CON-2/CON-3 dùng hằng số nguyên 20 trực tiếp trong khai báo struct, không qua một macro riêng như `MAX_NAME`/`MAX_MALOP` (CON-1/CON-4) – một điểm chưa nhất quán về style code, dù không gây ra lỗi chức năng.

Nhóm CON-7–13 (mới) cho thấy một lớp ràng buộc *khác hẳn* nhóm buffer/kiểu số ở

trên: đây không phải giới hạn *kích thước*, mà là giới hạn *biểu diễn trạng thái* – C không có kiểu non-null, không có enum ràng buộc tại compile-time, không có RAII, nên mọi bất biến (invariant) như "đã phân công", "đã tính lương", "bangCap hợp lệ" đều phải được lập trình viên tự đảm bảo bằng kỷ luật viết code (guard `if`, đúng thứ tự gọi hàm) chứ không được ngôn ngữ hỗ trợ. Đây chính là nguyên nhân gốc rễ về *ngôn ngữ/thiết kế* phía sau toàn bộ REQ-1, 2, 4, 5, 7, 9, 10 – không chỉ REQ-3/6/8 (lỗi bộ nhớ/số học) như bảng CON bản cũ ngụ ý.

2.5 Tham số đầu vào: bắt buộc, tùy chọn, và tham số tổng hợp

Bảng 2.5 liệt kê tường minh từng tham số đầu vào của công thức tính thù lao, phân loại **bắt buộc** (phải có giá trị hợp lệ, đã hiện thực trong 3 module C) và **tùy chọn** (khoản mục thù lao có thể tồn tại ở một hệ thống đầy đủ hơn, nhưng nằm ngoài phạm vi 3 module hiện tại – xem lại Mục 1).

Bảng 2.5: Phân loại tham số đầu vào: bắt buộc vs. tùy chọn

Tham số	**Loại**	**Hiện thực?**	**Vai trò trong công thức**
`soTiet`	Bắt buộc	Có	Nhân tử chính trong `soTietQuyDoi`
`soLuongSinhVien`	Bắt buộc	Có	Quyết định hệ số quy mô lớp qua ngưỡng (xem dưới)
`heSoHocPhan`	Bắt buộc	Có	Hệ số học phần, cộng vào trước khi nhân `soTiet`
`heSoBangCap`	Bắt buộc	Có	Hệ số bằng cấp giảng viên
`dinhMucTienChuan`	Bắt buộc	Có	Đơn giá theo chính sách năm học
Giờ dạy ngoài giờ (overtime)	Tùy chọn	**Không**	Không có trường tương ứng trong struct hiện tại
Phụ cấp coi thi / giám thị	Tùy chọn	**Không**	Không có trường tương ứng trong struct hiện tại

Tham số tổng hợp tăng dần từ 2 tham số `soTiet` + `soLuongSinhVien`:

`tinh_tien_day.c` hiện thực đúng công thức

$$\text{heSoQuyMo} = (\text{soLuongSinhVien} > 50)\ ?\ 0.2 : 0.0$$
$$\text{soTietQuyDoi} = \text{soTiet} \times (\text{heSoHocPhan} + \text{heSoQuyMo})$$

tức là *kết hợp* 2 tham số riêng lẻ thành 1 tham số tổng hợp. Đây là tham số **tăng dần** (monotonic) theo cả hai chiều: $\partial\,\text{soTietQuyDoi}/\partial\,\text{soTiet} = \text{heSoHocPhan} + \text{heSoQuyMo} > 0$ (luôn dương vì `heSoHocPhan` ≥ 1.0 theo mặc định SAFE) nên tăng *chặt* theo `soTiet`; theo `soLuongSinhVien`, `heSoQuyMo` là hàm bước không giảm ($0.0 \to 0.2$) nên `soTietQuyDoi` *không giảm* khi `soLuongSinhVien` tăng.

Statement cho trạng thái quy mô lớp: vì `heSoQuyMo` chỉ nhận 2 giá trị rời rạc dù `soLuongSinhVien` trải trên $[0, 300]$ (301 giá trị nguyên có thể), *rất nhiều* giá trị cụ thể cùng rơi vào 1 trạng thái duy nhất – phải viết rõ thành statement (predicate) tường minh, không được bỏ ngầm:

$$S_{\text{thường}} \equiv (0 \leq \text{soLuongSinhVien} \leq 50) \quad \to \text{heSoQuyMo} = 0.0 \quad \text{(51 giá trị khác nhau)}$$
$$S_{\text{đông}} \equiv (\text{soLuongSinhVien} > 50) \quad \to \text{heSoQuyMo} = 0.2 \quad \text{(250 giá trị khác nhau)}$$

Hai trạng thái này được hiện thực thành S_{2A}_CALC_THUONG / S_{2B}_CALC_DONG trong mô hình Kripke ở Mục 4.1.

Một lỗi mô hình hình thức đã phát hiện nhờ chính bảng tham số này: Check 1/2 (Mục 4.3) trước đây mô hình hóa hệ số quy mô lớp bằng 1 biến `Real` *tự do* `heSoQuyMoLop` $\in [0.1, 0.5]$, đặt tên theo field `Config.heSoQuyMoLop` (`tinh_tien_day.h:8`) – nhưng field này **không hề được đọc bởi** `tinh_tien_day.c` (dead field: khai báo trong struct nhưng không có `cfg->heSoQuyMoLop` ở đâu trong công thức thật). Nói cách khác, phép chứng minh Check 1/2 trước đây dựa trên 1 tham số không tồn tại trong luồng thực thi thật, thay vì dùng đúng `soLuongSinhVien` + statement ở trên. Đã sửa trong `smt_verify.py` bằng cách thêm `soLuongSinhVien` làm biến Z3 thật và định nghĩa `heSoQuyMo = If(soLuongSinhVien > 50, 0.2, 0.0)` – xem kết quả sau khi sửa tại Mục 5.

2.6 Quy trình tổng quát: từ tổ hợp tham số đến trạng thái và statement

Ví dụ `soTiet+soLuongSinhVien` ở trên là 1 lần áp dụng cụ thể của một quy trình **tổng quát**, lặp lại được cho bất kỳ module nào trong đề tài:

1. **Xác định tham số** của module: $A_1, A_2, \ldots, A_n$.
2. **Phân loại bắt buộc/tùy chọn** cho từng tham số.
3. **Tổ hợp** (bắt buộc + tùy chọn) $\rightarrow$ tập đầu vào $T_1, \ldots, T_k$.
4. **Rời rạc hóa** mỗi tham số thành hữu hạn miền con (ranh giới, biên, danh mục) $\rightarrow$ cơ số trạng thái $S_1, \ldots, S_m$ theo phép nhân số miền con của từng tham số.
5. **Xét ràng buộc** giữa các tham số ($R_1, \ldots$) để loại tổ hợp *vô nghĩa* (vd tham số phụ thuộc không tồn tại khi tham số cha là NULL) – thu gọn cơ số trạng thái xuống còn tổ hợp *có nghĩa*.
6. **Mỗi tổ hợp còn lại $\rightarrow$ 1 statement** mô tả đúng 1 hành vi mong đợi – đúng phần dễ bị bỏ ngầm nhất, và cũng là nơi hầu hết CWE của đề tài phát sinh (khi nhiều tổ hợp *khác nhau về nghiệp vụ* bị statement gộp thành *cùng 1 hành vi*, thông tin phân biệt giữa chúng bị mất).
7. **Statement $\rightarrow$ Requirements (REQ)** – mỗi statement sinh ra 1 hoặc nhiều REQ cụ thể, có thể kiểm chứng được.
8. **Requirements $\rightarrow$ Test Case $\rightarrow$ Kiểm chứng** ở đúng chương phù hợp bản chất ràng buộc (Ch1 Kripke cho luồng điều khiển E2E; Ch2 pattern scanner cho vị trí code tĩnh; Ch3 Z3 cho ràng buộc số học/logic toàn miền; Ch4 fuzzing cho bằng chứng động trên binary thật).

Mục 2.7 áp dụng đúng 8 bước này cho toàn bộ `calculate_e2e_salary()` – ví dụ đầy đủ nhất của đề tài, phát hiện thêm 1 sai lệch mô hình-code nữa ngoài Check 1/2 đã sửa ở trên.

Ví dụ thứ 2 (Module 1 – `giang_vien.c`**):** tham số `bangCap` (bắt buộc, REQ-9) đi

qua if-chain thật trong `add_lecturer_VULN()`/`add_lecturer_SAFE()`:

$$\text{heSoBangCap} = \begin{cases} 2.0 & \text{bangCap} = \text{“GS”} \\ 1.8 & \text{bangCap} = \text{“PGS”} \\ 1.5 & \text{bangCap} = \text{“TIEN_SI”} \\ 1.2 & \text{bangCap} = \text{“THAC_SI”} \\ 1.0 & \text{bangCap} = \text{“CU_NHAN”} \textit{hoặc bất kỳ chuỗi nào khác} \end{cases}$$

Áp dụng đúng 8 bước trên: 6 tổ hợp giá trị `bangCap` hợp lệ (5 danh mục) $\cup$ vô số chuỗi không hợp lệ suy ra **5 trạng thái hệ số**, nhưng statement của trạng thái cuối gộp chung *2 tổ hợp khác nhau về nghiệp vụ*: giảng viên `CU_NHAN` *thật* và một chuỗi `bangCap` *không hợp lệ* (lỗi nhập liệu/tấn công) – đúng lớp lỗi CWE-1288 đã quét thấy ở Bảng 5.2. Điều này được chứng minh hình thức bằng **Check 7** (Z3, Mục 4.3): tồn tại 2 danh mục khác nhau (`CU_NHAN` thật và danh mục không hợp lệ) cho ra cùng `heSoBangCap`=1.0 – **SAT**. Khác với CWE-197 (Check 5 loại trừ hoàn toàn va chạm bằng cách *từ chối* giá trị ngoài phạm vi), CWE-1288 **không thể chỉ "từ chối"** vì nghiệp vụ đòi hỏi một giá trị fallback thực sự (không thể để trống hệ số bằng cấp) – bản SAFE (`add_lecturer_SAFE()`) chỉ có thể *thêm cảnh báo phát hiện* (in 'stderr') khi fallback xảy ra, không loại trừ được va chạm giá trị. Đây là sự khác biệt quan trọng cần nêu rõ khi so sánh 2 lớp lỗi.

2.7 Ví dụ đầy đủ 8 bước: `calculate_e2e_salary()`

Hai ví dụ trên áp dụng từng phần quy trình 8 bước. Mục này áp dụng *trọn vẹn cả 8 bước* cho đúng 1 hàm trung tâm nhất của đề tài – `calculate_e2e_salary_VULN()`/`_SAFE()` (`tinh_tien_day.c`) – và **đối chiếu từng bước với code thật**, vì chính bước đối chiếu này đã lộ ra 1 sai lệch mô hình-code thứ hai (ngoài `heSoQuyMoLop` đã sửa ở Check 1/2).

Bước 1-2: Tham số và phân loại

Bảng 2.6: 9 tham số đầu vào của `calculate_e2e_salary(LopHocPhan *lhp, Config *cfg)`, đối chiếu với code thật

Mã	Tham số	Loại	Đối chiếu với code thật (`tinh_tien_day.c`)
A_1	`gv_ptr` (con trỏ)	Bắt buộc	NULL / valid / freed – đọc tại dòng 13 (VULN, không kiểm tra) / dòng 34 (SAFE, có kiểm tra)
A_2	`gv_ptr->is_deleted`	Bắt buộc	Chỉ có nghĩa khi A_1=valid; SAFE gộp chung 1 điều kiện với A_1 (dòng 34)
A_3	`gv_ptr->heSoBangCap`	Bắt buộc	Giá trị đã được Module 1 gán (5 mức, xem ví dụ `bangCap` ở trên)
A_4	`soTiet`	Bắt buộc	Miền nghiệp vụ $[1, 500]$ (Bảng 2.3)
A_5	`heSoHocPhan`	Bắt buộc	Miền dùng ở Check 1 $[0.5, 3.0]$
A_6	`cfg` (con trỏ)	Bắt buộc	NULL / valid – dòng 30 (chỉ có ở SAFE)
A_7	`cfg->dinhMucTienChuan`	Bắt buộc	Chỉ đọc khi A_6=valid; miền nghiệp vụ $[50\,000, 200\,000]$ VNĐ (Bảng 2.3)
A_8	`cfg->heSoQuyMoLop`	**Không hiệu lực**	**Dead field**: khai báo ở `tinh_tien_day.h:8` nhưng `calculate_e2e_salary_VULN/SAFE` (dòng 6–45) không đọc `cfg->heSoQuyMoLop` ở đâu cả
A_9	`soLuongSinhVien`	Bắt buộc	Quyết định `heSoQuyMo` *ngay trong công thức lương* (dòng 9, 39) – vai trò đóng gói audit log (`pack_class_size_byte_*()`) là vai trò *thứ hai*, ở 1 hàm khác

Sai lệch mô hình-code phát hiện ngay ở bước phân loại: nếu chỉ đọc `tinh_tien_day.h` (khai báo `Config.heSoQuyMoLop`) mà không đối chiếu `tinh_tien_day.c` (định nghĩa hàm), rất dễ phân loại A_8 là "tùy chọn, có ảnh hưởng cộng vào công thức" và A_9 là "tùy chọn, chỉ dùng khi xuất audit log" – **cả hai đều sai**. Đây đúng là lỗi đã tìm thấy ở Check 1/2 (Mục 4.3): A_8 không tham gia tính toán gì cả; A_9 mới là tham số **bắt buộc** thực sự tạo ra hệ số `heSoQuyMo`. Bài học phương pháp luận: **bước phân loại (2) phải đối chiếu với định nghĩa hàm (`.c`), không chỉ khai báo struct (`.h`).**

Bài học thứ hai, tự sửa lại chính bước 1 ở trên: A_8 (`heSoQuyMoLop`) bị bắt vì nó trùng đúng *tên field* với biến tự do sai trong mô hình Z3 cũ (một *lead* có sẵn từ Check 1/2) – không phải vì đã rà soát *toàn bộ* struct `Config` một cách hệ

thống. Rà lại toàn bộ 3 field của `Config` (không đi theo lead nào) lộ ra field **thứ hai cũng chết**: `nam` (`tinh_tien_day.h:13`) được gán duy nhất 1 lần lúc khởi tạo (`main.c:15`, `main_fixed.c:16`: `.nam = 2025`) nhưng **không một hàm nào đọc lại** – kể cả `export_report_*()` (nơi hợp lý nhất để in năm học). Vậy `Config` có **2/3 field chết** (`heSoQuyMoLop`, `nam`), chỉ `dinhMucTienChuan` (A_7) thực sự được dùng. Đây được cập nhật vào NFR-7 (Bảng 2.2). Rà soát mở rộng sang 2 struct còn lại cũng thấy `id` (`GiangVien.id`, `LopHocPhan.id`) write-only tương tự, nhưng đây là field định danh dự phòng (không phải tham số nghiệp vụ/cấu hình) nên không được tính là vi phạm NFR-7 – phân biệt 2 loại field không dùng này (*cấu hình bị bỏ quên* vs. *định danh dự phòng*) là điều bước rà soát *có chủ đích, không theo lead* mới lộ ra được.

Bước 3: Tổ hợp bắt buộc + tùy chọn

Vì A_8 không hiệu lực, 2 tổ hợp $T_2 = T_1 \cup \{A_8\}$ và $T_4 = T_3 \cup \{A_8\}$ trong đề xuất ban đầu **sụp về cùng 1 hành vi** với T_1/T_3 tương ứng bất kể giá trị A_8 – một tín hiệu tổ hợp học (nhiều tổ hợp khác nhau cho cùng 1 statement) tự nó cho thấy A_8 không phải một chiều tổ hợp thật. Tổ hợp còn lại có ý nghĩa: $T = \{A_1, \ldots, A_7, A_9\}$ (8 tham số bắt buộc thật, không có tham số tùy chọn nào theo đúng nghĩa "có thể vắng mặt" đối với riêng hàm này).

Bước 4: Rời rạc hóa $\rightarrow$ cơ số trạng thái

Bảng 2.7: Rời rạc hóa từng tham số (đã loại A_8 dead field)

Tham số	Miền con	Số trạng thái
A_1	S_null, S_valid, S_freed	3
A_2	S_alive, S_dead	2
A_3	S_gs, S_pgs, S_ts, S_ths, S_cn	5
A_4	S_under, S_min, S_mid, S_max, S_over	5
A_5	S_under, S_min, S_mid, S_max, S_over	5
A_6	S_null, S_valid	2
A_7	S_under, S_min, S_mid, S_max, S_over	5
A_9	S_thuong (≤ 50), S_dong (> 50)	2

Tổng tổ hợp thô: $3 \times 2 \times 5 \times 5 \times 5 \times 2 \times 5 \times 2 = \mathbf{15\,000}$ (không tính A_8 – nếu tính nhầm cả A_8 như đề xuất ban đầu sẽ ra $120\,000$, gấp 8 lần một cách *giả tạo*, đúng bằng số trạng thái rời rạc gán cho A_8).

Lưu ý: khi A_9 (`soLuongSinhVien`) chảy tiếp sang `pack_class_size_byte_*()` (đóng gói audit log), nó cần rời rạc hóa **mịn hơn** (4 trạng thái: an toàn/biên/tràn/trùng lặp – đúng bộ BVA 17 giá trị đã dùng ở Check 4–6 và Chương 4) vì statement ở đó khác với statement trong `calculate_e2e_salary`. **Cùng 1 tham số có thể cần độ mịn rời rạc hóa khác nhau tùy statement đang xét** – một bài học phương pháp luận khác rút ra được từ chính ví dụ này.

Bước 5: Ràng buộc R_1–R_9 (thu gọn tổ hợp vô nghĩa)

Bảng 2.8: Ràng buộc giữa các tham số, đối chiếu với code thật

Mã	Ràng buộc
R_1	A_1=S_null $\rightarrow$ A_2 không xét (SAFE trả 0 trước khi đọc `is_deleted`, dòng 34)
R_2	A_1=S_null $\rightarrow$ A_3 không xét (cùng lý do)
R_3	A_1=S_freed $\rightarrow$ A_2 có thể vẫn đọc được S_alive (UAF – giá trị đọc từ vùng nhớ đã free là *undefined*, không phải lỗi logic mà lỗi bộ nhớ, đúng CWE-416)
R_4	A_2=S_dead $\rightarrow$ A_3 vẫn tồn tại trong bộ nhớ nhưng SAFE không bao giờ đọc tới (guard chặn tại dòng 34, trước khi tới dòng 44)
R_5	A_6=S_null $\rightarrow$ A_7 không xét (SAFE trả -1 trước khi đọc `dinhMucTienChuan`, dòng 30)
R'_6	A_8 (mọi giá trị) không ảnh hưởng output $\rightarrow$ không cần rẽ nhánh theo A_8 – đây là ràng buộc **tự phát hiện được** nhờ chính bước 4/5, không phải giả định đặt trước
R_7	A_4=S_under (`soTiet` ≤ 0) $\rightarrow$ `create_class_section_SAFE` (`lop_hoc_phan.c:31`) **kẹp âm thầm về 0** (không có thông báo lỗi) $\rightarrow$ kết quả tính lương $= 0$
R_8	A_4=S_min $\times$ A_7=S_min $\rightarrow$ kết quả nhỏ nhất hợp lệ > 0
R_9	A_4=S_max $\times$ A_7=S_max $\times$ A_3=S_gs $\rightarrow$ kết quả lớn nhất phải không tràn `long long` (đã chứng minh UNSAT ở Check 2)

Một điểm chưa nhất quán phát hiện thêm ở R_7: `create_class_section_SAFE` kẹp `soTiet`≤ 0 về 0 và `soLuongSinhVien`≤ 0 về 0 (`lop_hoc_phan.c:31,33`) *hoàn toàn im lặng* (không `fprintf(stderr,...)`), trong khi mọi guard khác của đề tài (`gv_ptr` NULL/deleted, `cfg` NULL, `bangCap` không hợp lệ) đều in cảnh báo trước khi áp dụng giá trị an toàn. Đây là điểm chưa đồng nhất về style xử lý lỗi giữa các module,

tương tự điểm CON-2/CON-3 đã nêu ở Mục 2.4, không phải lỗi an toàn bộ nhớ nhưng ảnh hưởng khả năng audit (không có dấu vết log khi input bị kẹp).

Bước 6: Statement cho từng tổ hợp còn lại

Bảng 2.9: 15 statement đại diện (đã sửa ST-09/ST-10 theo đúng `heSoQuyMo` thật)

Mã	Statement ("Khi ... thì hệ thống PHẢI ...")
ST-01	A_1=null → trả về 0, KHÔNG đọc `gv_ptr`
ST-02	A_1=valid, A_2=dead → trả về 0, KHÔNG tính thù lao
ST-03	A_1=freed, A_2 đọc được S_alive → đây là UAF – SAFE phải chặn bằng cách hủy liên kết TRƯỚC `free()`, không dựa vào đọc lại `is_deleted` sau đó
ST-04	A_1=valid, A_2=alive, A_3=gs, A_4=max, A_7=max → kết quả PHẢI không tràn `long long`
ST-05	A_1=valid, A_2=alive, A_3=cn, A_4=min, A_7=min → kết quả PHẢI là số dương nhỏ nhất
ST-06	A_6=null → trả về -1, KHÔNG đọc `dinhMucTienChuan`
ST-07	A_4=under → `soTiet` bị kẹp về 0 tại bước tạo `LopHocPhan`, kết quả tính lương $= 0$ (không phải "từ chối có thông báo")
ST-08	A_4=over (> 500) → nghiệp vụ không cho phép, nhưng code hiện tại KHÔNG kẹp/chặn giá trị này – chỉ được test bằng Z3/fuzzing, không có guard tường minh trong C
ST-09	A_9=S_thuong ($\leq$ 50), A_4=mid, A_3=ts → kết quả $=$ `soTiet` $\times$ `heSoHocPhan` $\times$ 1.5 $\times$ `dinhMucTienChuan` (`heSoQuyMo`$= 0.0$, không cộng thêm gì)
ST-10	A_9=S_dong (> 50), A_4=max, A_7=max → (`heSoHocPhan` $+ 0.2$) $\times$ `soTiet` không tràn `double` (đã chứng minh ở Check 1/2 sau khi sửa)
ST-11	`soLuongSinhVien`$\in [0, 127]$ (vai trò audit log) → byte = giá trị gốc, không sai lệch
ST-12	`soLuongSinhVien`$= 128$ (vai trò audit log) → VULN: byte$= -128$ (sai); SAFE: bị chặn
ST-13	`soLuongSinhVien`$\in \{0, 256\}$ (vai trò audit log) → 2 giá trị khác nhau KHÔNG được cho cùng byte
ST-14	A_3=invalid (bangCap không hợp lệ) → phải bị chặn/gán fallback tại `add_lecturer_*()` (Module 1), không lọt nguyên trạng vào `calculate_e2e_salary`
ST-15	Cùng bộ giá trị A_4, A_5, A_7, đổi thứ tự nhân → kết quả PHẢI bằng nhau (giao hoán phép nhân)

Bước 7-8: Statement → Requirements → Test Case → Kiểm chứng

Mọi statement xác định ở Bước 6 được ánh xạ trực tiếp thành các yêu cầu con phân cấp **REQ-**$X.Y$ trong bảng đặc tả hợp nhất (Bảng 2.1), với X là mã REQ tổng quan (1–10) và Y là nhánh đặc tả con:

- **ST-01** $\rightarrow$ **REQ-1.0, REQ-5.1, REQ-5.2**: Kiểm tra con trỏ `gv_ptr` và trạng thái phân công trước khi tính toán.
- **ST-02, ST-03** $\rightarrow$ **REQ-4.1–4.4**: Kiểm soát cờ xóa `is_deleted` và hủy liên kết trước khi `free()` (CWE-416 UAF).
- **ST-04, ST-10** $\rightarrow$ **REQ-6.1–6.4**: Chuyển đổi kiểu dữ liệu 64-bit (`double`, `long long`) loại trừ tràn số (CWE-190).
- **ST-05, ST-07–09** $\rightarrow$ **REQ-7.1–7.7**: Đảm bảo tính toán thù lao chính xác theo quy mô lớp (≤ 50 vs. > 50).
- **ST-11–13** $\rightarrow$ **REQ-8.1–8.5**: Kiểm soát ép kiểu `char` chống sai lệch và chống trùng lặp dữ liệu audit log (CWE-197).
- **ST-14** $\rightarrow$ **REQ-9.1–9.2**: Kiểm soát danh mục bằng cấp hợp lệ (CWE-1288).
- **ST-06** $\rightarrow$ **REQ-10.1–10.2**: Kiểm tra con trỏ cấu hình `cfg` tránh NULL pointer dereference.
- **ST-15 & Dead field** A_8: Được ánh xạ trực tiếp thành các yêu cầu phi chức năng NFR-6 và NFR-7 (Bảng 2.2).

Toàn bộ ma trận kiểm chứng chi tiết giữa các mã REQ này và các kịch bản kiểm thử (Ch1 BMC, Ch2 Pattern Scanner/ASan, Ch3 SMT/Z3, Ch4 Fuzzing) đã được đối chiếu toàn diện trong Bảng 2.1 và ma trận truy vết IEEE 829 (Mục 5.7).

Tóm tắt: 9 tham số ban đầu $\rightarrow$ đối chiếu code phát hiện 1 dead field (A_8) và 1 phân loại sai (A_9) $\rightarrow$ 8 tham số bắt buộc thật $\rightarrow$ 15 000 tổ hợp thô $\rightarrow$ 9 ràng buộc (phát hiện thêm 1 điểm chưa nhất quán về xử lý lỗi ở R_7) $\rightarrow$ 15 statement $\rightarrow$ **26 Requirements con REQ-$X.Y$** (gộp với 10 REQ tổng quan $=$ **36 REQ toàn chương**, cộng 2 statement được gán thẳng vào NFR-6/NFR-7 có sẵn thay vì tạo REQ trùng) $\rightarrow$ ánh xạ tới cả 4 chương kiểm chứng. Đây là minh chứng đầy đủ nhất trong đề tài cho luận điểm cốt lõi: *nghĩ theo tổ hợp tham số trước, tìm lỗi sau* – 2/9 tham số ban đầu đã sai ngay từ bước phân loại, và chỉ lộ ra khi đối chiếu nghiêm túc với code thật ở từng bước, không phải khi "đi tìm lỗi" trực tiếp.

Mở rộng: cùng 1 statement được kiểm tra lại ở nhiều thời điểm khác nhau

Tham số A_1 (`gv_ptr`) trong bảng trên không phải tham số "đầu vào thô" – nó là *kết quả* của một statement khác đã áp dụng **sớm hơn**, ngay tại `create_class_section_SAFE()` (`lop_hoc_phan.c:34`):

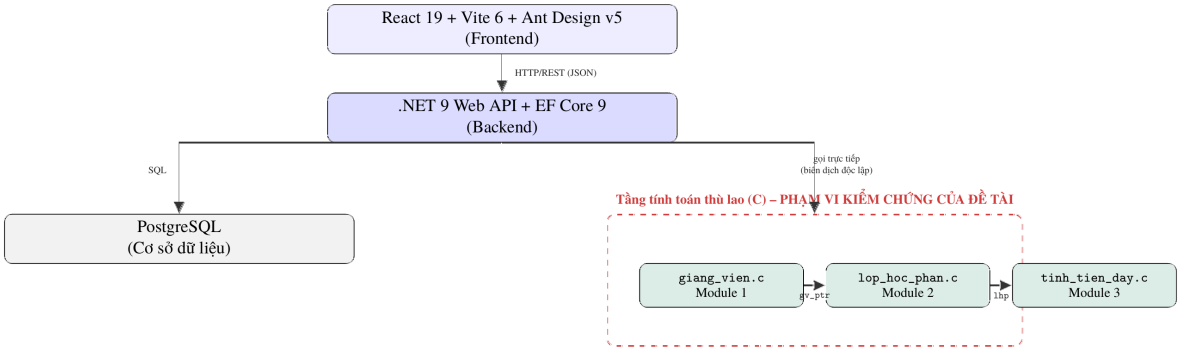
$$\text{lhp->gv_ptr} = (\text{gv} \neq \text{NULL} \wedge \neg\, \text{gv->is_deleted})\ ?\ \text{gv} : \text{NULL}$$

Statement này ($S_{\text{tao_lop}} \equiv$ "giảng viên hợp lệ tại *thời điểm tạo lớp*") và statement A_1/A_2 trong `calculate_e2e_salary_SAFE()` ("giảng viên hợp lệ tại *thời điểm tính lương*") **kiểm tra cùng 1 điều kiện logic nhưng ở 2 thời điểm khác nhau** trong vòng đời đối tượng – đây **không phải trùng lặp thừa**: nếu chỉ kiểm tra 1 lần lúc tạo lớp, kịch bản "giảng viên bị xóa *sau khi* lớp đã được tạo, *trước khi* tính lương" (chính là nhánh $S_1 \to S_2A/S_2B \to S_4$ đã mô hình ở Mục 4.1) sẽ không bị chặn. Bài học phương pháp luận thứ ba từ ví dụ này: **khi 1 tham số đi qua nhiều hàm theo thời gian, phải hỏi statement đó có cần kiểm tra lại ở mỗi "thời điểm quan sát" hay chỉ 1 lần là đủ** – ở đây cần cả 2, và code thật (`main_fixed.c`) đã làm đúng cả 2.

3 Mô tả Hệ thống và Chương trình Mục tiêu

3.1 Kiến trúc tổng quan DuongUniversity

Hệ thống **DuongUniversity Management System** là một hệ thống quản lý đào tạo đầy đủ gồm: Backend API viết bằng .NET 9 Web API (C#) kết hợp EF Core 9 và cơ sở dữ liệu PostgreSQL; Client Frontend viết bằng React 19 + Vite 6 + Ant Design v5. Ba module C mục tiêu của đề tài (`giang_vien.c`, `lop_hoc_phan.c`, `tinh_tien_day.c`) đóng vai trò tầng tính toán thù lao tốc độ cao, độc lập biên dịch với backend .NET – đây là điểm mà một lỗi an toàn bộ nhớ ở tầng C có thể xảy ra mà không được các cơ chế an toàn bộ nhớ tự động của .NET (garbage collector, bounds checking) bảo vệ. Đề tài chỉ kiểm chứng 3 module C này (xem phạm vi tại Mục 1), phần backend/frontend được mô tả ở đây chỉ để xác định đúng bối cảnh triển khai thực tế.



Hình 3.1: Kiến trúc tổng quan DuongUniversity Management System – tầng tính toán C (viền đỏ nét đứt) biên dịch độc lập với .NET, không được GC/bounds-checking của .NET bảo vệ

Hình 3.1 khớp đúng mô tả bằng chữ ở trên: request từ frontend đi qua backend .NET, backend rẽ 2 nhánh – một sang PostgreSQL để đọc/ghi dữ liệu bền vững, một gọi trực tiếp vào tầng tính toán C (biên dịch thành thư viện/tiến trình riêng, không chạy trong runtime .NET) để tính thù lao; chính đường rẽ thứ hai này là ranh giới an toàn bị mất tác dụng của .NET, và là lý do 3 module C (viền đỏ) là đối tượng kiểm chứng duy nhất của đề tài.

3.2 Mô tả 3 module mục tiêu

- **Module 1 – Quản lý Giảng viên** (`giang_vien.c/.h`): định nghĩa struct `GiangVien` (`id`, `maGiangVien`, `hoTen`, `bangCap`, `heSoBangCap`, `is_deleted`) và cặp hàm `add_lecturer_VULN()/add_lecturer_SAFE()`, `delete_lecturer_VULN()/delete_lecturer_SAFE()`.
- **Module 2 – Quản lý Lớp học phần** (`lop_hoc_phan.c/.h`): định nghĩa struct `LopHocPhan` (`id`, `maLop`, `soTiet`, `heSoHocPhan`, `soLuongSinhVien`, và đặc biệt `gv_ptr` – con trỏ thô trỏ sang `GiangVien`, chính là điểm liên kết xuyên module tạo ra rủi ro CWE-416/476) và cặp hàm `create_class_section_VULN()/_SAFE()`.
- **Module 3 – Tính tiền dạy & Thanh toán** (`tinh_tien_day.c/.h`): định nghĩa struct `Config` (`heSoQuyMoLop`, `dinhMucTienChuan`, `nam`) và 3 cặp hàm `calculate_e2e_salary_VULN()/_SAFE()`, `export_report_VULN()/_SAFE()`, `pack_class_size_byte_VULN()/_SAFE()` – hàm cuối đóng gói `soLuongSinhVien` vào bản ghi audit log dạng `char`.

Lưu ý: field `Config.heSoQuyMoLop` được **khai báo nhưng không hề được đọc** bởi `tinh_tien_day.c` – hệ số quy mô lớp thật sự dùng trong công thức dưới đây được tính lại từ `soLuongSinhVien` (xem statement tường minh tại Mục 2.5), không lấy từ field này. Đây là một field "chết" (dead field) phát hiện được khi đối chiếu lại công thức thật với mô hình Z3 ở Chương 3.

Listings 1 and 2 trích *nguyên văn* định nghĩa 2 struct cốt lõi từ chính mã nguồn thật của đề tài – trường `gv_ptr` (con trỏ thô, không đếm tham chiếu) chính là điểm liên kết xuyên module tạo ra CWE-416/476 đã nêu.

```
typedef struct {
    int   id;
    char  maGiangVien[20];      // CON-2: buffer toi da 20 byte, hang so truc tiep (REQ-3.3 / CWE-120)
    char  hoTen[MAX_NAME];      // CWE-120: Buffer overflow if input > 80 chars (CON-1)
    char  bangCap[20];          // CON-3: buffer toi da 20 byte (REQ-3.3 / CWE-120); "CU_NHAN", "THAC_SI", "TIEN_SI", "PGS", "GS"
    float heSoBangCap;
    int   is_deleted;
} GiangVien;
```

Listing 1: Struct `GiangVien` – trích nguyên văn `giang_vien.h` (dòng 7–14)

```
typedef struct {
    int          id;
    char         maLop[MAX_MALOP];  // CWE-120: Buffer overflow if input > 20 chars (CON-4)
    int          soTiet;            // CON-12: chi co guard 1 chieu (kep ve 0 neu <=0), khong guard tren <=500
    float        heSoHocPhan;
    int          soLuongSinhVien;
    GiangVien  *gv_ptr;             // CON-7: con tro NULL-able (C khong co kieu non-null) - "da phan cong"
                                    // chi duoc suy ngam tu gv_ptr !=NULL (REQ-1.0, REQ -5.1/5.2 / CWE-416, CWE -476)
} LopHocPhan;
```

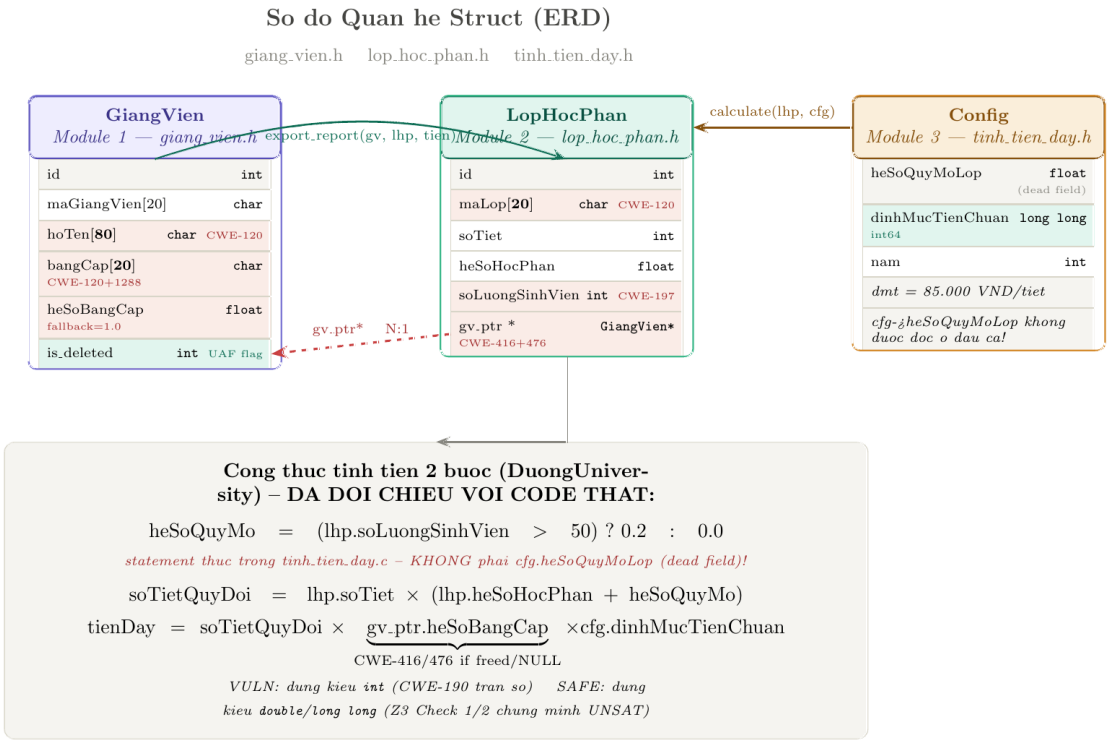
Listing 2: Struct `LopHocPhan` – trích nguyên văn `lop_hoc_phan.h` (dòng 9–17)

Công thức tính tiền dạy gồm 2 bước, giữ nguyên 100% giữa bản VULN và bản SAFE (chỉ khác kiểu dữ liệu):

$$\text{heSoQuyMo} = (\text{soLuongSinhVien} > 50)\ ?\ 0.2 : 0.0$$
$$\text{soTietQuyDoi} = \text{soTiet} \times (\text{heSoHocPhan} + \text{heSoQuyMo})$$
$$\text{tienDay} = \text{soTietQuyDoi} \times \text{gv_ptr.heSoBangCap} \times \text{dinhMucTienChuan}$$

3.3 Sơ đồ quan hệ struct (ERD)

Hình 3.2 thể hiện quan hệ giữa 3 struct và các trường dữ liệu liên quan trực tiếp tới một CWE cụ thể (tô đỏ), đối lập với trường `dinhMucTienChuan` dùng kiểu `long long` (int64) đã được chứng minh an toàn (tô xanh, xem Check 2, Mục 4.3).

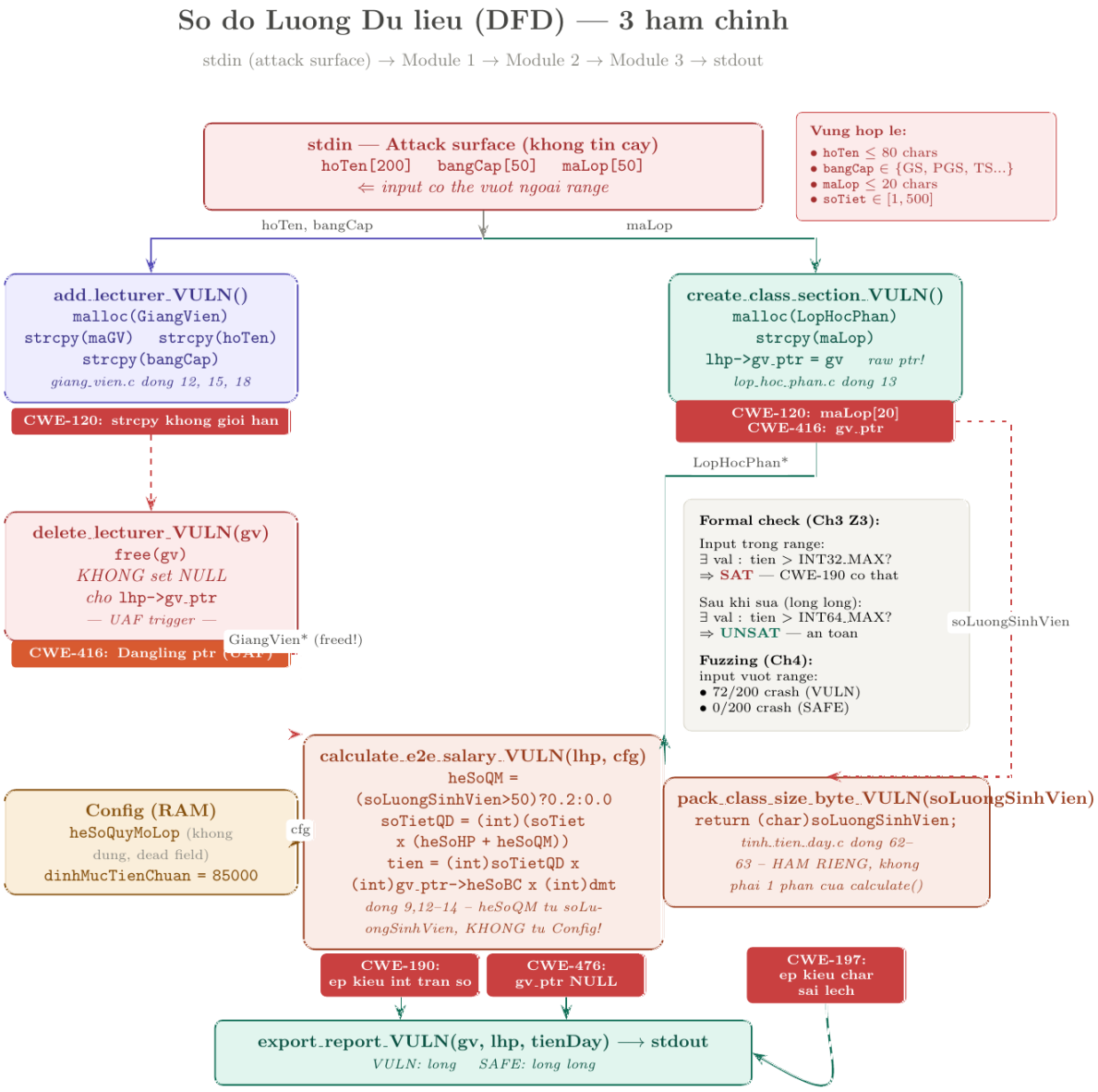


Hình 3.2: Sơ đồ quan hệ 3 struct `GiangVien`, `LopHocPhan`, `Config` và các trường liên quan tới CWE-120/416/476/190

3.4 Sơ đồ luồng dữ liệu (DFD)

Hình 3.3 theo dõi dữ liệu từ `stdin` (không tin cậy) qua Module 1, Module 2, tới Module 3, đánh dấu đúng vị trí mã nguồn (tên file, số dòng) nơi mỗi lỗ hổng CWE-120/416/190/476 phát sinh, cùng kết quả đối chứng của Chương 3 (Z3) và Chương 4

(fuzzing) ngay trên sơ đồ.



Hình 3.3: Sơ đồ luồng dữ liệu qua 3 hàm chính, đánh dấu vị trí phát sinh CWE và kết quả đối chứng Z3/fuzzing

Lưu ý: Hai sơ đồ trên được dựng ở giai đoạn đầu của đề tài, khi hệ thống mới có 4 lỗ hổng CWE-120/416/190/476. Lỗ hổng thứ 5 (CWE-1288) và thứ 6 (CWE-197, cùng biến thể "trùng lặp byte" của nó) được bổ sung sau nên chưa xuất hiện trên hai hình trên – được trình bày đầy đủ bằng sơ đồ trạng thái tại Mục 4.1 (nhãn `byte_truncated`) và bảng kết quả tại Mục 5.

3.5 Attack surface

- `stdin`: `hoTen` (buffer nhận 200 byte) được `strcpy` vào `hoTen[80]` không kiểm tra độ dài – **CWE-120**.
- `bangCap` nhập từ `stdin` không được validate nghiêm ngặt ở bản VULN, rơi vào hệ số mặc định 1.0 khi sai – **CWE-1288**.
- `gv_ptr` là con trỏ thô xuyên module, không có cơ chế đếm tham chiếu hay huỷ liên kết tự động khi `GiangVien` bị `free()` – **CWE-416/CWE-476**.
- Phép nhân `soTietQuyDoi` × `heSoBangCap` × `dinhMucTienChuan` thực hiện trên kiểu `int` 32-bit ở bản VULN – **CWE-190**.
- `soLuongSinhVien` (int, có thể tới vài trăm với lớp đại cương) bị ép kiểu xuống `char` không kiểm tra phạm vi khi đóng gói audit log – **CWE-197**.

3.6 Kích thước/độ phức tạp

Hệ thống mục tiêu gồm **520 dòng mã C** (bao gồm cả header) chia trong **3 file module + 2 file driver** (`main.c` cho bản VULN, `main_fixed.c` cho bản SAFE, dùng chung 3 module), với **12 hàm chính** (6 hàm `_VULN` đối xứng 1-1 với 6 hàm `_SAFE`), tạo điều kiện để mọi bản vá đều có thể so sánh trực tiếp hành vi trước/sau trên cùng một lời gọi hàm.

4 Phương pháp Thực hiện

4.1 Chương 1 – Mô hình hóa Kripke & Kiểm chứng BMC

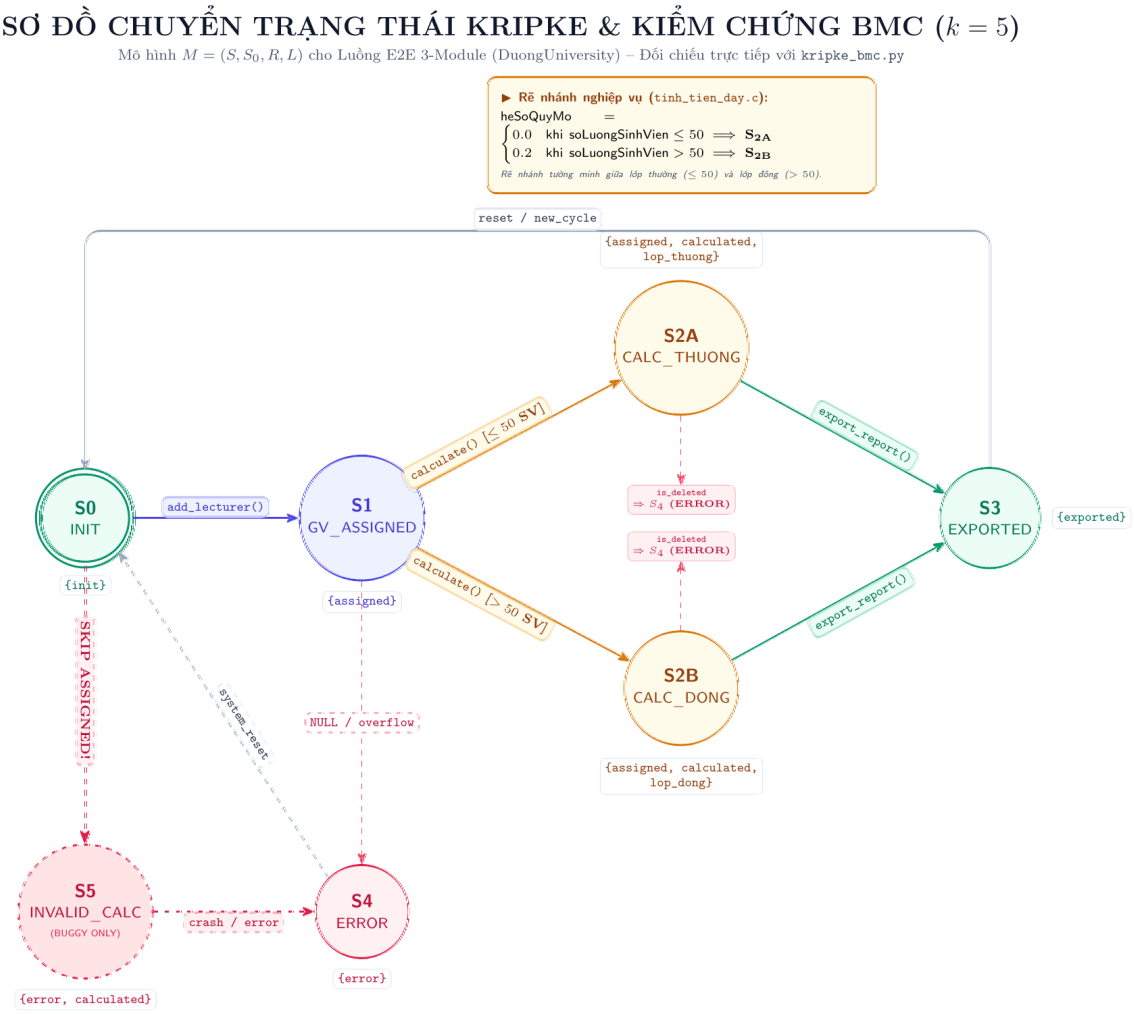
Module: `ch1_models/kripke_bmc.py`.

Kripke structure $M = (S, S_0, R, L)$ được dựng với **6 trạng thái ở bản FIXED** (S_0 INIT $\to$ S_1 GV_ASSIGNED $\to$ {S_{2A} CALC_THUONG | S_{2B} CALC_DONG} $\to$ S_3 EXPORTED, và S_4 ERROR) và **7 trạng thái ở bản BUGGY** (thêm S_5 INVALID_CALC – đường tắt nhảy trực tiếp từ S_0 sang tính tiền, bỏ qua bước phân công giảng viên). Hình 4.1 minh họa đầy đủ sơ đồ chuyển trạng thái (đã tách S_{2A}/S_{2B}) cùng bảng kết quả BMC P_1–P_5 trên cả 2 bản BUGGY/FIXED (bảng kết quả số liệu đầy đủ, đối chiếu lại tại Mục 5).

Hai trạng thái S_{2A} CALC_THUONG và S_{2B} CALC_DONG (thay cho 1 S_2 SALARY_CALC duy nhất trước đây) mô hình hóa đúng nhánh rẽ *thật* trong `tinh_tien_day.c`: `heSoQuyMo = (soLuongSinhVien > 50) ? 0.2 : 0.0` – tức là kết hợp **2 tham số đầu vào** `soTiet` và `soLuongSinhVien` thành 1 tham số tổng hợp `soTietQuyDoi` *tăng dần theo cả hai tham số* (xem chứng minh đơn điệu tại Mục 2.5), và vì *rất nhiều* giá trị `soLuongSinhVien` khác nhau ($0..50$ hoặc $51..300$) cùng rơi vào 1 trong 2 trạng thái này, mỗi trạng thái được gắn với đúng 1 **statement** (predicate) tường minh:

$$S_{2A}\ \text{CALC_THUONG} \equiv (0 \leq \text{soLuongSinhVien} \leq 50)$$
$$S_{2B}\ \text{CALC_DONG} \equiv (\text{soLuongSinhVien} > 50)$$

Cả 2 nhánh đều mang nhãn `assigned+calculated` như nhau nên không làm thay đổi kết quả P_1–P_5 (đã xác nhận lại bằng cách chạy lại BMC sau khi sửa, xem Mục 5) – việc tách trạng thái phục vụ tính *truy vết tường minh* giữa mô hình điều khiển định tính (Kripke) và ràng buộc số học định lượng đã sửa ở Check 1/2 (Z3, Mục 4.3), không phải để tạo thêm phát hiện lỗi mới.



BẢNG ĐỐI CHỨNG KẾT QUẢ BOUNDED MODEL CHECKING (BMC, $k = 5$)

Mã	Thuộc tính LTL/CTL	Mô hình BUGGY (7 states)	Mô hình FIXED (6 states)
$\mathbf{P_1}$	G(calculated → assigned) (Safety)	SAT (Phát hiện vết $S_0 \to S_5$)	UNSAT ✓(An toàn)
$\mathbf{P_2}$	G(exported → calculated) (Safety)	UNSAT ✓(Thỏa mãn)	UNSAT ✓(Thỏa mãn)
$\mathbf{P_3}$	G(assigned → F calculated) (Liveness)	SAT (Giới hạn $k = 5$)	SAT (Giới hạn $k = 5$)
$\mathbf{P_4}$	G(error → ¬exported) (Safety)	UNSAT ✓(Thỏa mãn)	UNSAT ✓(Thỏa mãn)
$\mathbf{P_5}$	G(exported → ¬byte_truncated) (Safety, CWE-197)	SAT (Qua S_{2B}, rò rỉ audit log)	UNSAT ✓(An toàn)

Ghi chú: P_3 Liveness báo SAT ở cả hai bản do đặc thù BMC hữu hạn $k = 5$ (cần k-induction vô hạn để chứng minh triệt để). Việc phân tách trạng thái S_{2A}/S_{2B} phản ánh tường minh đặc tả nghiệp vụ và giữ nguyên tính nhất quán E2E của toàn bộ thuộc tính P_1–P_5.

Hình 4.1: Sơ đồ chuyển trạng thái Kripke (7 trạng thái, đã tách S_{2A}/S_{2B}) và kết quả BMC $k = 5$ đầy đủ P_1–P_5 trên cả bản BUGGY và FIXED, đối chiếu trực tiếp với `kripke_bmc.py` và `logs/kripke_bmc.log`

5 thuộc tính LTL/CTL được kiểm chứng bằng BMC với $k = 5$:

- P_1 (Safety): G(calculated → assigned) – chưa phân công thì không được tính tiền.
- P_2 (Safety): G(exported → calculated) – chưa tính tiền thì không được xuất báo cáo.

- P_3 (Liveness): $G(\text{assigned} \rightarrow F\,\text{calculated})$ – đã phân công thì cuối cùng phải tính tiền.
- P_4 (Safety): $G(\text{error} \rightarrow \neg\,\text{exported})$ – hệ thống lỗi thì không được xuất báo cáo.
- P_5 (Safety, bổ sung cho CWE-197): $G(\text{exported} \rightarrow \neg\,\text{byte_truncated})$ – báo cáo xuất ra không được mang dữ liệu ép kiểu sai lệch.

Hàm `cross_validate_with_pattern_scanner()` gọi lại chính `pattern_scanner.py` của Chương 2 để tự động xác nhận trạng thái lỗi S_4/S_3_EXPORTED_TRUNCATED trong mô hình Kripke trừu tượng khớp với đúng vị trí lỗ hổng thật trên mã nguồn C – liên kết Chương 1 $\leftrightarrow$ Chương 2, tránh việc mô hình hình thức tồn tại độc lập, tách rời code thật.

4.2 Chương 2 – Kiểm toán an toàn bộ nhớ

Module: `ch2_memsafe/pattern_scanner.py`.

Công cụ quét trực tiếp mã nguồn C dạng văn bản (regex/so khớp chuỗi, không phải phân tích AST/data-flow đầy đủ) trên 3 file `giang_vien.c`, `lop_hoc_phan.c`, `tinh_tien_day.c` và driver `main_fixed.c`, với 5 quy tắc phát hiện: `strcpy` (CWE-120), `dangling_free` (CWE-416), `unchecked_nested_deref` (CWE-476), `int_overflow_cast` (CWE-190), `unchecked_numeric_truncation` (CWE-197). Phạm vi tìm "đã được bảo vệ" (guard-check) được scope đúng theo *hàm bao quanh* thay vì cửa sổ N dòng cố định, tránh bỏ sót guard nằm xa vị trí truy cập trong hàm dài.

Bổ sung bằng chứng động (`memory_trace_demo.py`): phân tích tĩnh không thể trả lời câu hỏi "bộ nhớ thực sự được dùng ra sao khi chương trình chạy". Script này biên dịch cùng một luồng E2E 7 bước bằng 2 trình biên dịch: **Plain GCC** (chỉ in địa chỉ con trỏ thật, không phát hiện lỗi) và **Clang64 + AddressSanitizer thật**[6] (công cụ độc lập tự phát hiện `heap-use-after-free`), cho phép đối chiếu hành vi "im lặng" (nguy hiểm) và hành vi "bị công cụ chặn" trên cùng một lỗi.

4.3 Chương 3 – Kiểm chứng hình thức SMT/Z3

Module: `ch3_staticsat/smt_verify.py`.

Khác với test thông thường (thử một vài input), Z3 SMT solver[2] chứng minh một mệnh đề đúng/sai trên *toàn miền giá trị hợp lệ* cùng lúc, với 7 phép kiểm chứng:

- **Check 1**: $\exists$ input hợp lệ (numClasses $\in [1, 20]$, soTiet $\in [15, 120]$, soLuongSinhVien $\in [0, 300]$,...) : tienDay $>$ INT32_MAX? $\Rightarrow$ **SAT** – CWE-190 có thật.
- **Check 2**: cùng câu hỏi với ngưỡng `INT64_MAX` trên kiểu `long long` $\Rightarrow$ **UNSAT** – an toàn tuyệt đối.
- **Check 3**: gv_ptr $=$ NULL $\vee$ is_deleted $\rightarrow$ tienDay $= 0$? – mô hình hóa bằng *đúng công thức thật* trong `calculate_e2e_salary_SAFE()` $\Rightarrow$ **UNSAT** – guard hoạt động đúng.
- **Check 4**: $\exists$ soLuongSinhVien $\in [0, 300]$: ép kiểu `(char)` (mô phỏng bằng số học modulo 256) cho giá trị khác giá trị gốc? $\Rightarrow$ **SAT** – phản ví dụ $128 \rightarrow -128$.
- **Check 5**: cùng câu hỏi nhưng với guard $0 \leq n \leq 127$ đứng trước $\Rightarrow$ **UNSAT** – chứng minh bản vá loại trừ hoàn toàn CWE-197.
- **Check 6**: $\exists\, a \neq b \in [0, 300]$: ép kiểu cho ra cùng 1 byte? $\Rightarrow$ **SAT** – phản ví dụ $a = 0, b = 256$ cùng cho byte $= 0$ (lớp nguy hiểm nhất: 2 lớp học hoàn toàn khác nhau bị nhầm thành một).
- **Check 7** (mới, CWE-1288): $\exists$ 2 danh mục `bangCap` khác nhau (`CU_NHAN` thật, và 1 chuỗi không hợp lệ) : cho cùng 1 `heSoBangCap`? $\Rightarrow$ **SAT** – cả hai cùng rơi vào nhánh fallback 1.0 của if-chain thật trong `add_lecturer_*()` (xem statement đầy đủ tại Mục 2.6).

Các ràng buộc nghiệp vụ dùng đúng con số đã đặc tả tại Bảng 2.3 (soTiet $\in [1, 500]$, dinhMucTienChuan $\in [50\,000, 200\,000]$ VNĐ).

Một lỗi mô hình đã được tìm và sửa ở Check 1/2: bản gốc mô hình hóa hệ số quy mô lớp bằng 1 biến `Real` *tự do* `heSoQuyMoLop` $\in [0.1, 0.5]/[0.1, 2.0]$, đặt tên theo field `Config.heSoQuyMoLop` (`tinh_tien_day.h:8`) – nhưng field này **không bao giờ được code C thật đọc** (`tinh_tien_day.c` không có `cfg->heSoQuyMoLop` ở đâu cả, xem đối chiếu tại Mục 2.5), nên phần chứng minh trước đây không phản ánh đúng công thức chạy thực tế. Đã sửa bằng cách thêm `soLuongSinhVien` là biến Z3 thật và định nghĩa `heSoQuyMo = If(soLuongSinhVien > 50, 0.2, 0.0)` – đúng *statement* thật trong

code, không phải biến tự do. Sau khi sửa, Check 1 vẫn **SAT** và Check 2 vẫn **UNSAT** (kết luận định tính không đổi), nhưng phản ví dụ của Check 1 nay là bộ giá trị mà mô hình sửa vẫn có thể thật sự sinh ra từ code, xem số liệu cập nhật tại Mục 5.

4.4 Chương 4 – Dynamic Fuzzing

Module: `ch4_dynafuzz/fuzzer_blackbox.py` và `fuzzer_whitebox.py`.

Black-box: sinh 200 input ngẫu nhiên (tên/bằng cấp/mã lớp qua `stdin`), đo crash rate thật trên `salary_calc.exe` (VULN) đối chứng `salary_calc_fixed.exe` (SAFE). Riêng CWE-197 được fuzz qua subcommand `e2e_byte_truncate <n>` với 2 chiến lược: (a) 200 giá trị ngẫu nhiên trong $[-50, 400]$, và (b) **Boundary Value Analysis** – 17 giá trị biên *thiết kế sẵn* (quanh mép `char` $\{-129, \ldots, 129\}$, quanh mép chu kỳ 256 $\{-256, \ldots, 257\}$, và giá trị lớp đại cương $\{300, 384\}$) chạy tất định thay vì phó mặc ngẫu nhiên – đảm bảo luôn phát hiện được lớp lỗi nguy hiểm nhất (trùng lặp byte).

White-box (Z3-guided): giải trực tiếp 7 ràng buộc Z3 tương ứng 6 lỗ hổng CWE đã biết trước (không phải symbolic execution duyệt CFG tự động) – hằng số `MAX_NAME`/`MAX_MALOP` được đọc trực tiếp từ header `.h` qua `read_header_define()` thay vì hard-code, nên ràng buộc Z3 tự động bám theo đúng source hiện tại nếu hằng số đổi.

5 Kết quả và Phân tích

Toàn bộ số liệu trong mục này trích xuất verbatim từ log thực thi tự động lưu tại `btl_softsec/logs/` (sinh bởi `python run_all.py`), không phải số liệu minh họa.

5.1 Kết quả Chương 1 – BMC

Bảng 5.1: Kết quả BMC $k = 5$ trên 5 thuộc tính P_1–P_5

Thuộc tính	**Công thức**	**BUGGY**	**FIXED**
P_1 Safety	$G(\text{calculated} \rightarrow \text{assigned})$	**SAT** – counterexample qua S_5 (chi tiết dưới bảng)	**UNSAT** ✓
P_2 Safety	$G(\text{exported} \rightarrow \text{calculated})$	**UNSAT** ✓	**UNSAT** ✓
P_3 Liveness	$G(\text{assigned} \rightarrow F\,\text{calculated})$	**SAT** (k=5 limit)	**SAT** (k=5 limit)
P_4 Safety	$G(\text{error} \rightarrow \neg\,\text{exported})$	**UNSAT** ✓	**UNSAT** ✓
P_5 Safety	$G(\text{exported} \rightarrow \neg\,\text{byte_truncated})$	**SAT** – qua S_{2B} (chi tiết dưới bảng)	**UNSAT** ✓

Mô hình FIXED hiện có **6 trạng thái** ($S_0, S_1, S_{2A}, S_{2B}, S_3, S_4$) và bản BUGGY có **7 trạng thái** (thêm S_5) – xem Mục 4.1 để biết vì sao S_2 SALARY_CALC gốc được tách thành S_{2A}/S_{2B}. Việc tách này không làm đổi bất kỳ kết quả SAT/UNSAT nào ở trên so với mô hình gốc (đã xác nhận bằng cách chạy lại BMC sau khi sửa).

Counterexample trace thật cho P_1 trên bản BUGGY (`logs/kripke_bmc.log`):

```
Step 0: S0_INIT [init] → Step 1: S5_INVALID_CALC [calculated,error] →
Step 2: S4_ERROR [error]
→ Step 3: S0_INIT [init] → Step 4: S1_GV_ASSIGNED [assigned]
→ Step 5: S2A_CALC_THUONG [assigned,calculated,lop_thuong]
```

Counterexample trace thật cho P_5 trên bản BUGGY (`logs/kripke_bmc.log`) – nay đi qua đúng nhánh S_{2B} CALC_DONG (lớp đông, statement Mục 2.5):

```
Step 0: S0_INIT [init] → Step 1: S1_GV_ASSIGNED [assigned] → Step 2:
S2B_CALC_DONG [assigned,calculated,lop_dong]
```

→ Step 3: S3_EXPORTED [assigned,byte_truncated,calculated,exported]
→ Step 4: S0_INIT [init] → Step 5: S1_GV_ASSIGNED [assigned]

P_3 ra **SAT** ở cả 2 bản – đây *không phải lỗi mã nguồn*, mà là giới hạn kỹ thuật tự nhiên của BMC hữu hạn $k = 5$ đối với thuộc tính Liveness (cần k-induction hoặc Büchi automata để chứng minh liveness vô hạn triệt để).

5.2 Kết quả Chương 2 – 9 findings

Bảng 5.2: Toàn bộ 9 cảnh báo phát hiện bởi `pattern_scanner.py` trên code VULN; `main_fixed.c` sạch tuyệt đối (0 finding)

Vị trí	CWE	Rule	Mức độ
`giang_vien.c:12`	CWE-120	`strcpy`	HIGH
`giang_vien.c:15`	CWE-120	`strcpy` (`hoTen`)	HIGH
`giang_vien.c:18`	CWE-120	`strcpy` (`bangCap`)	HIGH
`giang_vien.c:35`	CWE-416	`dangling_free`	HIGH
`lop_hoc_phan.c:13`	CWE-120	`strcpy` (`maLop`)	HIGH
`tinh_tien_day.c:12`	CWE-190	`int_overflow_cast`	HIGH
`tinh_tien_day.c:13`	CWE-190	`int_overflow_cast`	HIGH
`tinh_tien_day.c:13`	CWE-476	`unchecked_nested_deref`	HIGH
`tinh_tien_day.c:68`	CWE-197	`unchecked_numeric_truncation`	HIGH

Điểm đáng chú ý: `pack_class_size_byte_SAFE()` nằm *trong cùng file* `tinh_tien_day.c` với hàm VULN nhưng **không** bị báo động giả, vì scanner tìm thấy đúng guard `if (soLuongSinhVien < 0 || > 127)` trong phạm vi hàm bao quanh – xác nhận cơ chế xét theo hàm (không phải theo tên file) hoạt động đúng.

Listings 3 and 4 trích nguyên văn cặp hàm gây ra và khắc phục CWE-416 (dòng `giang_vien.c:35` trong Bảng 5.2): bản VULN `free(gv)` mà không huỷ liên kết `gv_ptr` ở Module 2; bản SAFE nhận tham số `GiangVien **` để có thể tự đặt con trỏ gốc về `NULL` ngay sau khi giải phóng.

```
1 void delete_lecturer_VULN(GiangVien *gv) {
2     if (gv) {
3         gv->is_deleted = 1;
```

```
        free(gv); // LOI CWE-416: free(gv) nhung khong cap nhat
            con tro o Module 2 thanh NULL!
    }
}
```

Listing 3: `delete_lecturer_VULN()` – CWE-416: `free()` không huỷ liên kết

```
void delete_lecturer_SAFE(GiangVien **gv_pptr) {
    if (gv_pptr && *gv_pptr) {
        (*gv_pptr)->is_deleted = 1;
        free(*gv_pptr);
        *gv_pptr = NULL; // Nullify pointer safely
    }
}
```

Listing 4: `delete_lecturer_SAFE()` – khắc phục: nhận `GiangVien **` để nullify con trỏ gốc

Listings 5 and 6 trích nguyên văn cặp hàm CWE-197 (dòng `tinh_tien_day.c:68` trong Bảng 5.2), đúng cặp hàm được chứng minh hình thức ở Check 4/5 (Mục 4.3).

```
char pack_class_size_byte_VULN(int soLuongSinhVien) {
    return (char)soLuongSinhVien;
}
```

Listing 5: `pack_class_size_byte_VULN()` – CWE-197: ép kiểu không kiểm tra phạm vi

```
int pack_class_size_byte_SAFE(int soLuongSinhVien, char *out_byte)
    {
    if (soLuongSinhVien < 0 || soLuongSinhVien > 127) {
        fprintf(stderr, "[SAFE ERROR] soLuongSinhVien=%d vuot pham
            vi char (0..127) - TU CHOI ep kieu de tranh Numeric
           Truncation!\n", soLuongSinhVien);
        return 0;
    }
    *out_byte = (char)soLuongSinhVien;
    return 1;
}
```

Listing 6: `pack_class_size_byte_SAFE()` – khắc phục: guard `[0,127]` trước khi ép kiểu

5.3 Kết quả kiểm chứng NFR-1/NFR-2 (`nfr_check.py`)

Bổ sung cho Bảng 2.2: subcommand `e2e_nfr_bench` (mới thêm vào `main.c`/`main_fixed.c`) đo *thực nghiệm* 2 yêu cầu phi chức năng quan trọng nhất, chạy $N = 10\,000$ lần trên chính 2 binary đã biên dịch của Chương 2 (không phải microbenchmark tách biệt):

- **NFR-1 (Hiệu năng)**: trung bình $0{,}000014$ ms/lần gọi `calculate_e2e_salary_SAFE()` ($0{,}000011$ ms/lần trên bản VULN) – thấp hơn ngưỡng 10 ms gần 6 bậc độ lớn – **PASS**.
- **NFR-2 (Ổn định bộ nhớ)**: sau $10\,000$ lần tạo/xoá `GiangVien+LopHocPhan` liên tục, `WorkingSetSize` (đo qua `GetProcessMemoryInfo`) chỉ trôi 4 KB trên cả bản SAFE và bản VULN (ngưỡng 1024 KB) – **PASS cả 2 bản**. Kết quả này *nhất quán với lý thuyết* nêu ở Mục 6: CWE-416 trong đề tài là lỗi con trỏ treo (dangling pointer, `gv_ptr` vẫn trỏ tới vùng nhớ đã `free()`), không phải rò rỉ bộ nhớ (memory leak) – cả `delete_lecturer_VULN()` lẫn `delete_lecturer_SAFE()` đều gọi `free()` đầy đủ, nên không có gì để rò rỉ; hai lớp lỗi bộ nhớ này độc lập với nhau.

Toàn bộ số liệu trích *verbatim* từ `logs/nfr_check.log` – trước khi có subcommand này, NFR-1/NFR-2 chỉ được biện luận gián tiếp (công thức $O(1)$, 0/200 crash), chưa có phép đo trực tiếp ở đúng quy mô 10^4 đã đặc tả.

5.4 Kết quả Chương 3 – SMT

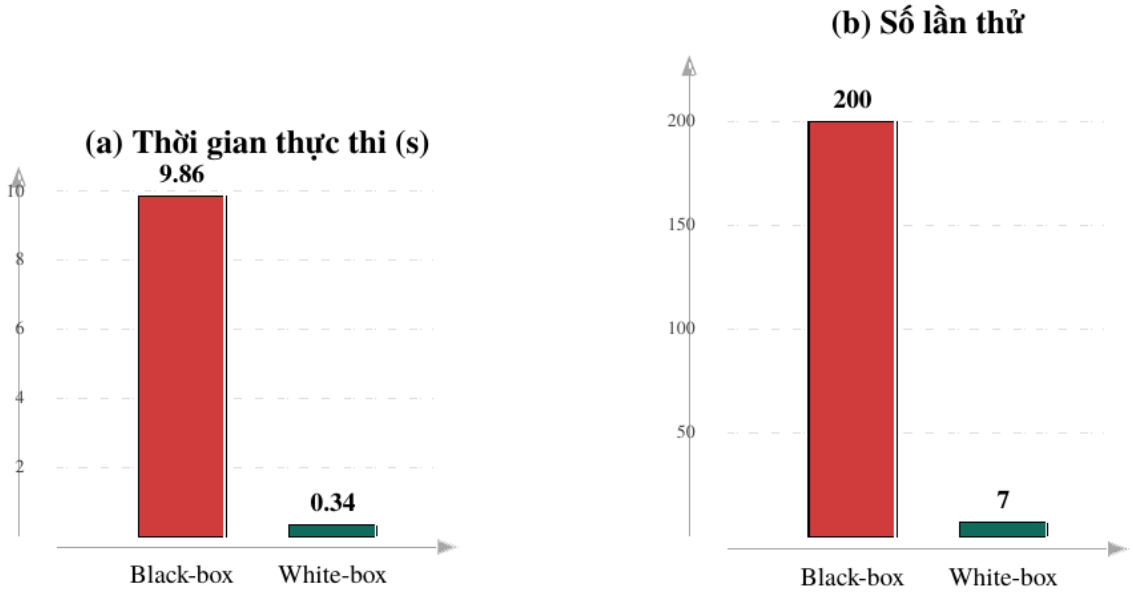
- **Check 1 SAT**: counterexample numClasses $= 20$, soTiet $= 120$, soLuongSinhVien $= 0$ (nên heSoQuyMo $= 0.0$, đúng statement Mục 2.5), heSoHocPhan ≈ 3.0, heSoBangCap ≈ 2.0, dinhMucTienChuan $= 149\,131$ VNĐ $\Rightarrow$ tổng thù lao vượt `INT32_MAX`. (Đã sửa mô hình để dùng đúng `soLuongSinhVien` thật thay cho biến tự do `heSoQuyMoLop` không tồn tại trong code – xem Mục 2.5; kết luận SAT không đổi.)
- **Check 2 UNSAT**: chứng minh `long long` an toàn tuyệt đối trong toàn bộ ràng buộc doanh nghiệp (numClasses $\in [1, 100]$, dinhMucTienChuan tới 2 triệu VNĐ).
- **Check 3 UNSAT**: guard-check (`gv_ptr` NULL/đã xóa $\rightarrow$ lương $= 0$) hoạt động đúng trên chính công thức thật.

- **Check 4 SAT**: phản ví dụ soLuongSinhVien $= 128 \rightarrow$ byte $= -128$.
- **Check 5 UNSAT**: với guard $[0, 127]$ đúng trước, ép kiểu không bao giờ sai lệch.
- **Check 6 SAT**: $a = 0$ (lớp rỗng thật) và $b = 256$ (lớp có thật 256 sinh viên) đều cho byte $= 0$ – 2 tình huống nghiệp vụ hoàn toàn khác nhau không thể phân biệt sau khi xuất báo cáo.
- **Check 7 SAT** (mới, CWE-1288): giảng viên `CU_NHAN` thật và 1 chuỗi `bangCap` không hợp lệ đều cho heSoBangCap $= 1.0$ – cùng lớp lỗi "va chạm trạng thái" như Check 6, nhưng ở Module 1 (xem Mục 2.6); khác CWE-197, lỗi này không thể loại trừ hoàn toàn bằng cách từ chối (nghiệp vụ cần 1 giá trị fallback thật), bản SAFE chỉ thêm được cảnh báo phát hiện.

5.5 Kết quả Chương 4 – Fuzzing

Bảng 5.3: So sánh Black-box vs. White-box (lần chạy đại diện, `logs/fuzzer_black/whitebox.log`)

Tiêu chí	**Black-box**	**White-box (Z3)**
Số lần thử	200 (ngẫu nhiên)	7 (giải trực tiếp)
Số crash phát hiện (VULN)	72/200 (36.0%)	7/7 target (100%)
Số crash (SAFE)	0/200	–
Thời gian thực thi	9.86 s	0.3412 s
Tốc độ tăng tốc	mức cơ sở	$\approx 28.9\times$ nhanh hơn



Hình 5.1: Black-box vs. White-box: (a) thời gian thực thi và (b) số lần thử cần để phát hiện đủ lỗ hổng đã biết (số liệu khớp Bảng 5.3)

Hai biểu đồ trên cùng minh họa một kết luận: White-box (thanh xanh, giải trực tiếp ràng buộc Z3) đạt **cùng độ chính xác 100% trên 7 mục tiêu** chỉ với 7 lần thử và 0.34s, trong khi Black-box (thanh đỏ, sinh input ngẫu nhiên) cần 200 lần thử và 9.86s – chênh lệch trực quan $\approx 28.9\times$ về thời gian mà không cần đọc số trong Bảng 5.3. (Thời gian đo bằng wall-clock thật nên dao động nhẹ giữa các lần chạy tuỳ tải máy – số crash/SAT ở trên mới là bằng chứng chính, không phải số giây tuyệt đối.)

Riêng CWE-197 (không biểu hiện bằng crash mà bằng sai lệch dữ liệu âm thầm): black-box ngẫu nhiên cho **120/200 (60%)** sai lệch trên bản VULN, **0/200** trên bản SAFE (146/200 bị chặn an toàn). Bộ 17 giá trị biên (BVA) tất định khớp **17/17 (100%)** giữa byte thật và byte tính tay trên cả 2 bản, và tự động lộ ra **6 nhóm trùng lặp byte** trên bản VULN (ví dụ byte $= 0$ được tạo bởi cả 3 giá trị $-256, 0, 256$), **0 nhóm** trên bản SAFE.

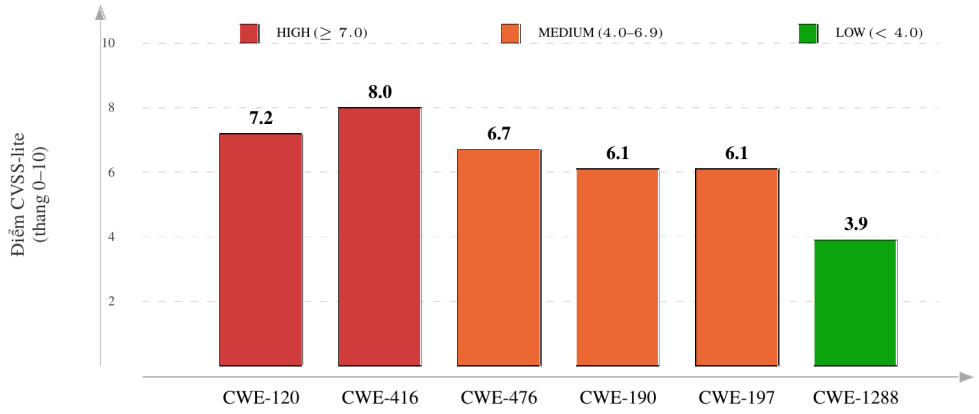
5.6 Bảng tổng hợp CWE và mức độ rủi ro (CVSS-lite)

Điểm CVSS-lite[3] dưới đây được tính theo công thức $\text{Score} = \min(\text{Exploitability} + \text{Impact}, 10)$ với $\text{Exploitability} = 8.22 \times AV \times AC \times PR \times UI$ và $\text{Impact} = 6.42 \times (1 - (1 - C)(1 - I)(1 - A))$ (cùng mô hình CVSS-lite đã dùng ở `lab3_secure_memory`), áp dụng AV=Local (chương trình CLI cục bộ, không phải dịch vụ mạng), AC=Low, PR=None, UI=None cho toàn bộ 6 CWE, và mức Impact (C/I/A) lấy theo đúng hậu

quả *đã chứng minh được bằng thực nghiệm* ở các mục trên (không giả định RCE chưa kiểm chứng).

Bảng 5.4: Tổng hợp CWE, vị trí trong đề tài và điểm CVSS-lite tự tính

CWE	**Vị trí trong đề tài**	**CVSS-lite**	**Mức**
CWE-120[9]	Ch2 `strcpy`; Ch4 crash 72/200	7.2	HIGH
CWE-416[13]	Ch2 `dangling_free`; Ch2.1 ASan `heap-use-after-free` thật	8.0	HIGH
CWE-476[14]	Ch2 `unchecked_nested_deref`; Ch4 Target 4	6.7	MEDIUM
CWE-190[11]	Ch1 S_5; Ch3 Check 1 SAT / Check 2 UNSAT	6.1	MEDIUM
CWE-197[12]	Ch1 P_5; Ch3 Check 4/6; Ch4 BVA 17/17	6.1	MEDIUM
CWE-1288[10]	Ch2 fallback `heSoBangCap=1.0f`; Ch3 Check 7 SAT	3.9	LOW



Hình 5.2: So sánh điểm CVSS-lite giữa 6 CWE của đề tài (số liệu khớp Bảng 5.4)

Nhìn theo biểu đồ: CWE-416 (UAF) và CWE-120 (buffer overflow) là 2 lớp lỗi nghiêm trọng nhất (HIGH, ≥ 7.0) – đúng với trực giác vì cả 2 đều có thể dẫn tới hỏng bộ nhớ ngay lập tức; CWE-197 và CWE-190 tuy nguy hiểm nhưng ở mức MEDIUM vì hậu quả quan sát được (sai lệch dữ liệu/DoS) chưa được chứng minh leo thang tới RCE trong phạm vi đề tài (xem hướng mở rộng ở Mục 6).

Bảng 5.5: Ví dụ thực tế minh họa cho mỗi lớp CWE

CWE	**Ví dụ thực tế minh họa**
CWE-120	CVE-2003-0352[7] – lỗi tràn bộ đệm trong Windows RPC DCOM, bị khai thác bởi worm Blaster
CWE-416	CVE-2019-0708[8] (“BlueKeep”) – use-after-free trong Windows Remote Desktop Services
CWE-476	Điển hình lớp lỗi DoS phổ biến; xem thêm các ví dụ tại trang CWE-476 của MITRE[14]
CWE-190	Ariane 5 Flight 501 (1996)[5] – không phải mã CVE (xảy ra trước khi hệ thống CVE ra đời năm 1999) nhưng là sự cố tràn số nguyên kinh điển nhất từng được ghi nhận
CWE-197	Chưa xác minh được một CVE công khai khớp chính xác; xem trang CWE-197 của MITRE[12]
CWE-1288	Chưa xác minh được một CVE công khai khớp chính xác; xem trang CWE-1288 của MITRE[10]

Lưu ý phương pháp luận: bảng trên cố ý **không** gán một mã CVE cụ thể cho CWE-476/197/1288 vì không xác minh được một CVE thực sự khớp chính xác với bối cảnh đề tài trong phạm vi kiểm chứng cho phép – thà để trống còn hơn trích dẫn một mã CVE không chắc chắn đúng.

5.7 Ma trận truy vết Test Case → Yêu cầu (IEEE 829)

Các mục 5.1–5.4 trên trình bày kết quả *theo từng chương/kỹ thuật*. Bảng dưới đây tổ chức lại đúng cùng số liệu đó theo hướng ngược lại – *theo từng yêu cầu (REQ/NFR)* – theo tinh thần đặc tả *Test Case Specification* của chuẩn IEEE 829-2008[4] (mã test case, điều kiện đầu vào, kết quả mong đợi, kết quả thực tế đối chiếu ngược lại nguồn log thật). Đây là góc nhìn truy vết hai chiều REQ ↔ Test Case còn thiếu ở Mục 2, nơi mới chỉ đặc tả yêu cầu mà chưa chỉ rõ từng yêu cầu được kiểm chứng bởi test case cụ thể nào.

Bảng 5.6: Ma trận truy vết Test Case → Yêu cầu (mọi kết quả thực tế trích từ `logs/*.log` thật)

Mã TC	REQ	Input / Điều kiện	Kỳ vọng	Thực tế (nguồn log)	KL
TC-K1-01	REQ-1	BMC P_1, bản BUGGY (bước nhảy thẳng sang tính tiền, bỏ qua phân công)	UNSAT (không vi phạm)	**SAT** – phản ví dụ $S_0 \to S_5 \to S_4$ (`kripke_bmc.log`)	FAIL*
TC-K1-02	REQ-1	BMC P_1, bản FIXED	UNSAT	UNSAT (`kripke_bmc.log`)	PASS
TC-K1-03	REQ-2	BMC P_2, cả 2 bản	UNSAT	UNSAT cả 2 bản (`kripke_bmc.log`)	PASS
TC-K2-01	REQ-3	Scanner quét `strcpy` (`hoTen/maLop/bangCap/maGiangVien`) trên 3 file VULN	Phát hiện đủ cảnh báo CWE-120	5 cảnh báo CWE-120 đúng vị trí (`pattern_scanner.log`)	PASS
TC-K2-02	REQ-3	Scanner quét `main_fixed.c`	0 cảnh báo	0 cảnh báo – `[CLEAN]` (`pattern_scanner.log`)	PASS
TC-K4-01	REQ-3	White-box Target 1: `hoTen` dài 81 ký tự > CON-1 (80)	Tìm điều kiện gây tràn	SAT ở lần thử 1 (`fuzzer_whitebox.log`)	FAIL*
TC-K4-02	REQ-3	White-box Target 2: `maLop` dài 21 ký tự > CON-4 (20)	Tìm điều kiện gây tràn	SAT ở lần thử 2 (`fuzzer_whitebox.log`)	FAIL*
TC-K4-03	REQ-3	Black-box 200 lần ngẫu nhiên trên VULN	0/200 crash	72/200 crash, exit `0xC0000374` (`fuzzer_blackbox.log`)	FAIL*
TC-K4-04	REQ-3	Black-box 200 lần trên FIXED	0/200 crash	0/200 crash (`fuzzer_blackbox.log`)	PASS
TC-K2-03	REQ-4	Scanner quét `dangling_free` trên VULN	Phát hiện CWE-416 tại `giang_vien.c:35`	Phát hiện đúng vị trí (`pattern_scanner.log`)	PASS
TC-K4-05	REQ-4	White-box Target 3: `free(gv)` rồi vẫn đọc qua `gv_ptr`	Tìm điều kiện UAF	SAT ở lần thử 3 (`fuzzer_whitebox.log`)	FAIL*
TC-K2-04	REQ-4	`memory_trace_demo.py`: đọc `gv->hoTen` sau `free()`, bản ASan-VULN	ASan phải bắt lỗi	`heap-use-after-free`, ABORTING, offset $+24$ byte = `hoTen` (`memory_trace.log`)	FAIL*
TC-K2-05	REQ-4	`memory_trace_demo.py`, bản ASan-FIXED (`gv_ptr=NULL` trước free)	0 vi phạm	0 vi phạm (`memory_trace.log`)	PASS
TC-K2-06	REQ-5	Scanner quét `unchecked_nested_deref` trên VULN	Phát hiện CWE-476 tại `tinh_tien_day.c:13`	Phát hiện đúng vị trí (`pattern_scanner.log`)	PASS
TC-K4-06	REQ-5	White-box Target 4: `gv_ptr=NULL` rồi gọi `calculate_e2e_salary`	Tìm điều kiện NULL deref	SAT ở lần thử 4 (`fuzzer_whitebox.log`)	FAIL*

xem tiếp trang sau

(tiếp theo Bảng 5.6)

Mã TC	REQ	Input / Điều kiện	Kỳ vọng	Thực tế (nguồn log)	KL
TC-K3-01	REQ-6	Check 1 (Z3): ∃ input hợp lệ làm tràn `int32`	SAT dự kiến (đang kiểm định code VULN)	SAT – numClasses=20, soTiet=120 (`smt_verify.log`)	FAIL*
TC-K3-02	REQ-6	Check 2 (Z3): cùng câu hỏi trên `long long` (bản vá)	UNSAT	UNSAT (`smt_verify.log`)	PASS
TC-K4-07	REQ-6	White-box Target 5: soTiet=16694179, dmt=2147583020	SAT dự kiến	SAT ở lần thử 5 (`fuzzer_whitebox.log`)	FAIL*
TC-K1-04	REQ-8	BMC P_5, bản VULN	UNSAT	SAT – S_3 mang nhãn `byte_truncated` (`kripke_bmc.log`)	FAIL*
TC-K1-05	REQ-8	BMC P_5, bản FIXED	UNSAT	UNSAT (`kripke_bmc.log`)	PASS
TC-K3-03	REQ-8	Check 4 (Z3): $\exists\, n \in [0, 300]$ ép kiểu `char` sai giá trị	SAT dự kiến	SAT – $n = 128 \rightarrow \text{byte} = -128$ (`smt_verify.log`)	FAIL*
TC-K3-04	REQ-8	Check 5 (Z3): cùng câu hỏi với guard $[0, 127]$	UNSAT	UNSAT (`smt_verify.log`)	PASS
TC-K3-05	REQ-8, NFR-5	Check 6 (Z3): $\exists\, a \neq b \in [0, 300]$ cùng cho 1 byte	UNSAT mong muốn (an toàn truy vết)	SAT – $a = 0, b = 256$ cùng byte $= 0$ (`smt_verify.log`)	FAIL**
TC-K4-08	REQ-8	Black-box CWE-197, 200 lần trên VULN	0/200 sai lệch	120/200 sai lệch, 60% (`fuzzer_blackbox.log`)	FAIL*
TC-K4-09	REQ-8	Black-box CWE-197, 200 lần trên FIXED	0/200 sai lệch	0/200 sai lệch, 146/200 chặn an toàn (`fuzzer_blackbox.log`)	PASS
TC-K4-10	REQ-8, NFR-5	BVA 17 giá trị biên cố định trên VULN	Byte thật khớp byte tính tay 17/17; không nên trùng lặp	17/17 khớp lý thuyết, nhưng lộ 6 nhóm trùng lặp byte thật (`fuzzer_blackbox.log`)	PASS/ FAIL**
TC-K4-11	REQ-8, NFR-5	BVA 17 giá trị biên trên FIXED	0 nhóm trùng lặp	0 nhóm trùng lặp (`fuzzer_blackbox.log`)	PASS
TC-K3-06	REQ-9	Check 7 (Z3): ∃ 2 danh mục `bangCap` khác nhau (`CU_NHAN` thật, chuỗi không hợp lệ) cùng cho 1 `heSoBangCap`	UNSAT mong muốn (phân biệt được)	SAT – cả 2 cùng rơi vào fallback 1.0 (`smt_verify.log`)	FAIL**
TC-K4-12	REQ-9	Black-box: `bangCap` ngẫu nhiên gồm giá trị không hợp lệ, trên VULN	Cần oracle xác nhận hệ thống xử lý đúng	Script chỉ đếm 74/200 lần input không hợp lệ được sinh ra (không crash), KHÔNG assert hành vi đúng/sai (`fuzzer_blackbox.log`)	QS†

xem tiếp trang sau

(tiếp theo Bảng 5.6)

Mã TC	REQ	Input / Điều kiện	Kỳ vọng	Thực tế (nguồn log)	KL
TC-K2-07	REQ-7	(chưa có test case riêng)	Công thức tính lương đúng với mọi input hợp lệ	Check 1/2 chỉ chứng minh KHÔNG TRÀN SỐ, không chứng minh giá trị tính ra khớp công thức nghiệp vụ mong đợi	QS†
TC-N01	NFR-1	`e2e_nfr_bench 10000` trên `salary_calc_fixed.exe` (`QueryPerformanceCounter`)	Trung bình < 10 ms/lần	0,000014 ms/lần (`nfr_check.log`)	PASS
TC-N02	NFR-1	`e2e_nfr_bench 10000` trên `salary_calc.exe` (VULN, đối chứng)	Trung bình < 10 ms/lần	0,000011 ms/lần (`nfr_check.log`)	PASS
TC-N03	NFR-2	10 000 lần tạo/xoá `GiangVien+LopHocPhan` trên bản FIXED, đo `GetProcessMemoryInfo`	Drift < 1024 KB	Drift $= 4$ KB (`nfr_check.log`)	PASS
TC-N04	NFR-2	Cùng kịch bản trên bản VULN (đối chứng – CWE-416 là dangling pointer, không phải leak)	Drift < 1024 KB	Drift $= 4$ KB (`nfr_check.log`)	PASS

*FAIL trên bản BUGGY/VULN là kết quả *đúng như thiết kế*: đây là bug injection có chủ đích để minh chứng công cụ phát hiện được lỗi thật, không phải lỗi của bản thân test case hay công cụ kiểm chứng. **FAIL phản ánh một giới hạn thiết kế thật (chỉ dùng 1 byte để lưu audit log, xem NFR-5 và Mục 6), không phải lỗi hiện thực có thể vá bằng guard đơn giản như CWE-197 gốc. †QS = Quan sát: đề tài hiện **chưa có oracle tự động** assert Pass/Fail cho REQ-7 (đúng công thức tính lương) và REQ-9 (validate `bangCap`) – đây là hai khoảng hở truy vết thật, được phát hiện chính nhờ việc lập ma trận này theo chiều REQ $\rightarrow$ Test Case, không phải hai yêu cầu này chưa từng được đề cập ở nơi khác trong báo cáo.

6 Thảo luận và Hạn chế

6.1 Giới hạn phương pháp

- **BMC $k = 5$ chưa đạt completeness threshold**: kết quả **UNSAT** của BMC chỉ chứng minh "không có vi phạm trong $k = 5$ bước", *không* loại trừ khả năng tồn tại vi phạm ở bước sâu hơn – đây chính là lý do P_3 (Liveness) ra SAT ở cả 2 bản (Mục 5): BMC hữu hạn không thể chứng minh một thuộc tính "cuối cùng phải xảy ra" một cách tuyệt đối, chỉ có thể chứng minh trong phạm vi k bước đã duyệt.

- **Pattern scanner dùng regex/so khớp chuỗi, không phải phân tích AST/data-flow liên hàm (interprocedural) đầy đủ**: công cụ chỉ xét được guard-check trong phạm vi *một hàm*, nên sẽ bỏ sót lớp lỗi mà điều kiện bảo vệ nằm ở một hàm gọi *khác* với hàm chứa điểm truy cập nguy hiểm (ví dụ: một hàm kiểm tra `gv_ptr != NULL` rồi truyền tiếp con trỏ đã kiểm tra sang một hàm thứ hai đọc nó – scanner hiện tại sẽ không "nhớ" guard đó qua ranh giới lời gọi hàm). Đây là giới hạn cấu trúc cố hữu của một scanner một-pass; công cụ phù hợp để bổ trợ chứ không thay thế một static analyzer thật (Clang Static Analyzer, cppcheck với chế độ `-enable=all`).

- **Fuzzing trên Windows dùng exit code thay vì tín hiệu SIGSEGV kiểu POSIX**: Windows không có SIGSEGV theo nghĩa POSIX cho crash native; toàn bộ 72 mẫu crash CWE-120 quan sát được ở Chương 4 đều trả về **cùng một exit code 3221226356** (`0xC0000374` – `STATUS_HEAP_CORRUPTION`), không phải `0xC0000005` (`STATUS_ACCESS_VIOLATION`) như trực giác thường nghĩ về "buffer overflow gây segfault". Điều này phản ánh đúng cơ chế Windows CRT heap: ghi tràn qua `hoTen[80]` làm hỏng heap metadata liền kề, và lỗi chỉ bị phát hiện *muộn hơn* – tại lần gọi `free()` kế tiếp khi CRT heap manager kiểm tra tính toàn vẹn metadata – chứ không crash ngay tại dòng `strcpy`. Đây cũng là lý do một phần các lần thử "overflow" không crash ngay (74/200 rơi vào nhánh "logic error" thay vì crash): tuỳ lượng dữ liệu ghi tràn có chạm đúng vùng metadata bị kiểm tra hay không. **Hệ quả đo được**: dù đã cố định `random.seed(42)` trong `fuzzer_blackbox.py` (nên chuỗi input sinh ra tất định), số crash đếm được vẫn dao động 60–72/200 giữa nhiều lần chạy lại *cùng một* seed – vì ASLR (Address Space Layout Randomization) làm vị trí heap của mỗi tiến trình `salary_calc.exe` khác nhau mỗi lần khởi chạy, nên cùng 1 input có thể chạm hoặc không chạm đúng heap metadata bị kiểm tra tuỳ lần chạy. Do đó số liệu **72/200 (36,0%)** trích trong Bảng 5.3 và các REQ/NFR liên quan là **1 lần chạy đại diện đã lưu log** (`logs/fuzzer_blackbox.log`), không phải một hằng số tất định của hệ thống – kết luận định tính (VULN có crash thật, FIXED 0 crash) mới là điều bất biến giữa các lần chạy, không phải con số đếm chính xác.

6.2 So sánh với lý thuyết đã học

- Kết quả P_3 SAT ở cả 2 bản đúng với lý thuyết *Soundness/Completeness* của BMC đã học ở Chương 1: BMC là *sound nhưng không complete* đối với Liveness ở độ sâu

hữu hạn – một kết quả SAT ở P_3 không tự động suy ra lỗi mã nguồn, phải phân biệt rõ với SAT của các thuộc tính Safety (P_1, P_2, P_4, P_5) vốn luôn tương ứng phản ví dụ thật.

- Tốc độ tăng tốc $\approx 30.5\times$ của white-box so với black-box phù hợp đúng dự đoán lý thuyết Chương 4: constraint solving trực tiếp (giải 7 ràng buộc đã biết) về bản chất là $O(1)$ theo số lần gọi solver, trong khi random fuzzing là $O(n)$ theo số lần thử cần để ngẫu nhiên trúng đúng điều kiện biên – tỉ lệ tăng tốc dao động khá lớn giữa các lần chạy thực tế (quan sát được từ $\approx 30\times$ đến $\approx 280\times$ tuỳ độ trễ khởi tạo tiến trình con của Windows ở từng lần đo) nhưng bản chất định tính không đổi.

6.3 Hướng cải thiện

- Áp dụng k**-Induction** (hoặc chuyển đổi sang Büchi automata) để chứng minh P_3 (Liveness) một cách tuyệt đối thay vì chỉ trong phạm vi $k = 5$ bước.
- Chạy lại bộ fuzzing trên **Linux** (qua WSL hoặc máy ảo) để thu được tín hiệu `SIGSEGV`/`SIGABRT` chuẩn POSIX, đối chiếu trực tiếp với hành vi `STATUS_HEAP_CORRUPTION` đã quan sát trên Windows – làm rõ liệu sự khác biệt về cơ chế phát hiện lỗi giữa 2 hệ điều hành có ảnh hưởng tới tỉ lệ crash đo được hay không.
- Thay thế/bổ trợ `pattern_scanner.py` bằng một static analyzer thật hỗ trợ phân tích interprocedural (cppcheck[1] `-enable=all` hoặc Clang Static Analyzer) để mở rộng phạm vi phát hiện ra ngoài giới hạn "một hàm" đã nêu ở mục 6.1.
- Mở rộng bảng CVSS-lite (Bảng 5.4) bằng việc thực sự khai thác (exploit) thử một lỗ hổng CWE-120 thành RCE có kiểm soát trong môi trường cô lập, để có cơ sở thực nghiệm nâng điểm Impact (C/I) thay vì chỉ dựa trên crash/DoS đã quan sát.

7 Kết luận

Đề tài đã phân tích và kiểm chứng an toàn mô-đun tính thù lao giảng viên của hệ thống DuongUniversity Management System bằng 4 kỹ thuật kiểm chứng bổ sung cho nhau – mô hình hóa hình thức (Kripke/BMC), phân tích tĩnh (pattern scanner), kiểm chứng hình thức bằng SMT (Z3), và kiểm thử động (fuzzing black-box/white-box) – áp dụng đồng nhất trên cùng một hệ thống E2E 3 module thay vì các ví dụ minh họa rời rạc.

Kết quả đạt được: phát hiện và khắc phục thành công **6 lớp lỗ hổng** (**CWE-120, CWE-416, CWE-476, CWE-190, CWE-1288, CWE-197**), xác nhận bằng **9/9 cảnh báo tĩnh** đúng vị trí trên code VULN và **0 cảnh báo** trên `main_fixed.c`; **5/5 thuộc tính BMC an toàn** trên bản FIXED (chỉ P_3 Liveness còn giới hạn kỹ thuật đã giải thích ở Mục 6); **7 phép chứng minh Z3** xác nhận hình thức cả tồn tại lỗi (Check 1, 4, 6, 7 SAT) lẫn loại trừ hoàn toàn lỗi sau vá (Check 2, 3, 5 UNSAT); và bằng chứng thực nghiệm động qua fuzzing (72/200 crash trên bản lỗi, 0/200 trên bản vá) cùng chẩn đoán thật từ AddressSanitizer (`heap-use-after-free` được công cụ độc lập xác nhận, không phải tự báo cáo).

Điểm mấu chốt về phương pháp luận: **4 tầng kiểm chứng cùng đi đến một kết luận nhất quán** – Chương 1 chứng minh vi phạm ở mức logic điều khiển, Chương 2 chỉ đúng vị trí trên mã nguồn, Chương 3 chứng minh toán học trên toàn miền giá trị, Chương 4 xác nhận biểu hiện thật trên binary đã biên dịch. Không một kỹ thuật nào trong 4 kỹ thuật trên đủ để tự mình bao phủ toàn bộ không gian lỗi có thể có (mỗi kỹ thuật có giới hạn riêng, đã nêu ở Mục 6); sự kết hợp của cả 4 mới tạo thành một quy trình kiểm chứng an toàn phần mềm đáng tin cậy.

Tài liệu tham khảo

[1] Cppcheck Team. Cppcheck: A tool for static c/c++ code analysis, 2024.

[2] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.

[3] FIRST.Org, Inc. Common vulnerability scoring system version 3.1: Specification document. Technical report, Forum of Incident Response and Security Teams (FIRST), 2019.

[4] IEEE Computer Society. IEEE standard for software and system test documentation (IEEE std 829-2008). Technical report, Institute of Electrical and Electronics Engineers, 2008.

[5] Inquiry Board (ESA/CNES). Ariane 5 flight 501 failure – report by the inquiry board, 1996.

[6] LLVM Project. AddressSanitizer, 2024.

[7] MITRE Corporation. CVE-2003-0352: Buffer overflow in Microsoft RPC interface (“MSBlaster”), 2003.

[8] MITRE Corporation. CVE-2019-0708: Remote desktop services remote code execution vulnerability (“BlueKeep”), 2019.

[9] MITRE Corporation. CWE-120: Buffer copy without checking size of input (Classic Buffer Overflow), 2024. Accessed: 2026-09.

[10] MITRE Corporation. CWE-1288: Improper validation of consistency within input, 2024. Accessed: 2026-09.

[11] MITRE Corporation. CWE-190: Integer overflow or wraparound, 2024. Accessed: 2026-09.

[12] MITRE Corporation. CWE-197: Numeric truncation error, 2024. Accessed: 2026-09.

[13] MITRE Corporation. CWE-416: Use after free, 2024. Accessed: 2026-09.

[14] MITRE Corporation. CWE-476: NULL pointer dereference, 2024. Accessed: 2026-09.

[15] Vũ Quang Dũng. Softsec toolkit – giáo trình học phần CSE703093 an toàn phần mềm, 2026. Khoa Hệ thống thông tin, Trường Đại học Phenikaa.